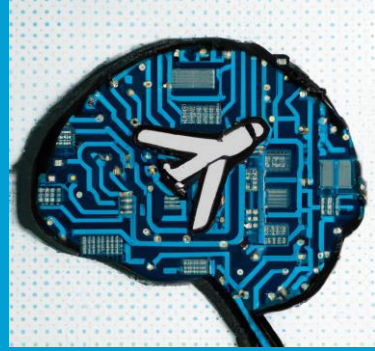
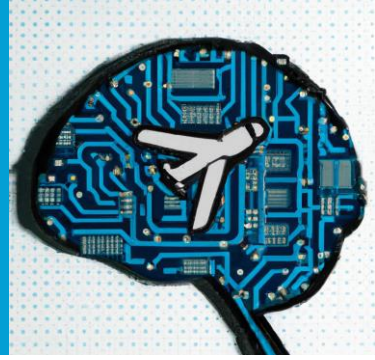




Introduction to AI

AE4353

Course Instructors



Lectures



Guido de Croon
g.c.h.e.decroon@tudelft.nl



Marija Popovic
M.Popovic@tudelft.nl



Christophe De Wagter
c.dewagter@tudelft.nl

Practical sessions



Quentin Missinne
Q.Missinne@student.tudelft.nl

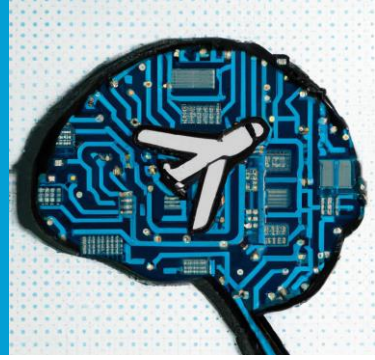


Reinier Vos
R.W.Vos@tudelft.nl



Dequan Ou
D.Ou@tudelft.nl

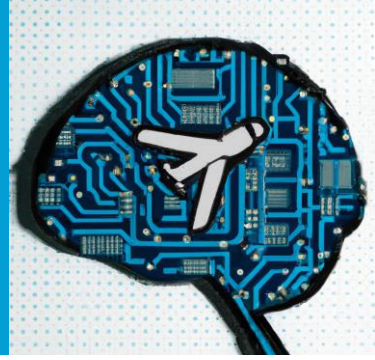
Lecture objectives



Students can:

- Describe the course setup and the overview of material covered in the course.
- Can explain machine learning terms such as “i.i.d. assumptions”, “inductive bias”, and “hypothesis space” and their relevance to machine learning problems.

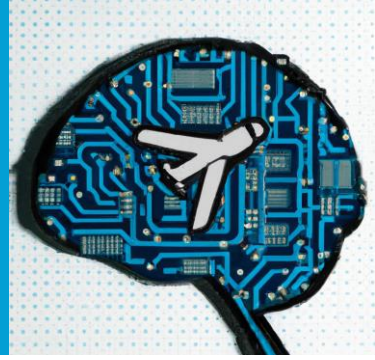
Learning objectives



Students can:

- Implement the delta learning rule for a perceptron and indicate a perceptron's limitations.
- Explain how the backpropagation algorithm for training multi-layer perceptrons works.

Deep neural network feats



Winning ImageNet
with large margins

AlexNet
Image: 224 (height) x 224 (width) x 3 (channels)
Convolution with 11x11 kernel + 4 stride 54x54x96
ReLU
Pool with 3x3 max kernel + 2 stride 26x26x96
Convolution with 5x5 kernel + 2 stride 26x26x128
ReLU
Pool with 3x3 max kernel + 2 stride 12x12x128
Convolution with 3x3 kernel + 1 pad 12x12x128
ReLU
Convolution with 3x3 kernel + 1 pad 12x12x128
ReLU
Convolution with 3x3 kernel + 1 pad 12x12x256
ReLU
Pool with 3x3 max kernel + 2 stride 5x5x256
Flatten
Dense: 4096 fully connected neurons
ReLU, dropout p=0.5
Dense: 4096 fully connected neurons
ReLU, dropout p=0.5
Dense: 1000 fully connected neurons
Output: 1 of 1000 classes

*Krizhevsky et al.
Comms ACM,
2017.*

Beating top-notch
StarCraft II players



*Vinyals, O. et al.
Nature, 2019.*

ChatGPT released



2012

2016

2019

2022

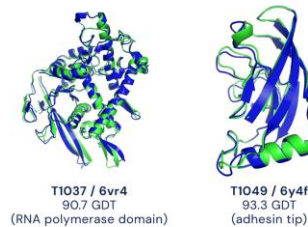
2023



Beating world
champion Go

*Silver, D. et al.
Nature, 2016.*

2021



● Experimental result
● Computational prediction

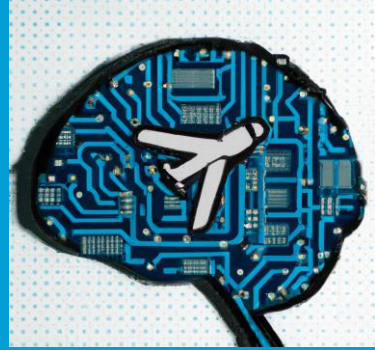
Setting a new standard in
protein folding

*Jumper, J. et al.,
Nature 2021.*

*Kaufmann, E. et
al. Nature, 2023.*



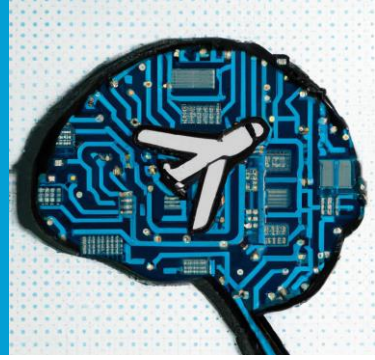
Beating champion-level
drone racers



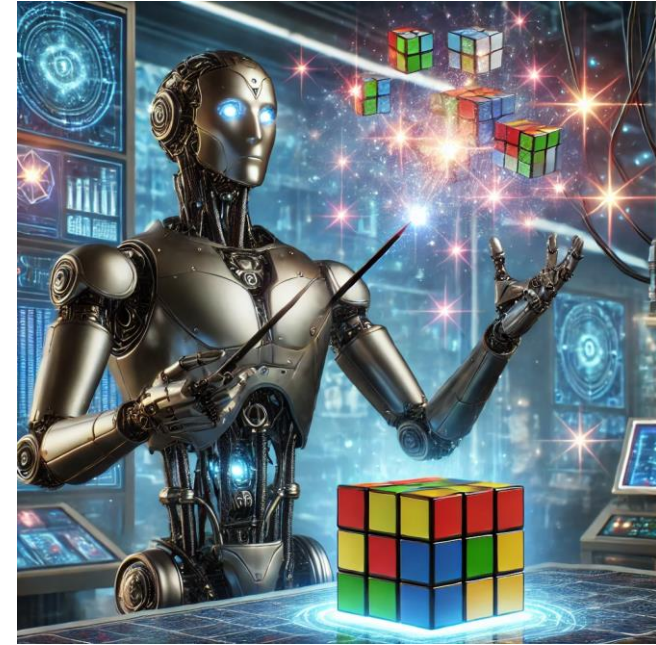
AE 4353

Artificial Intelligence for Aerospace Control and Operations

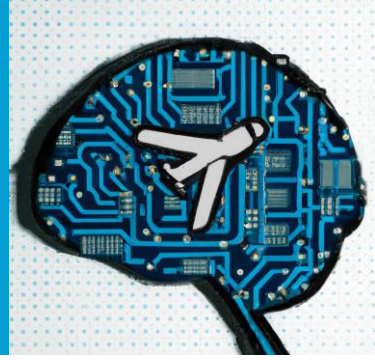
Course Goals



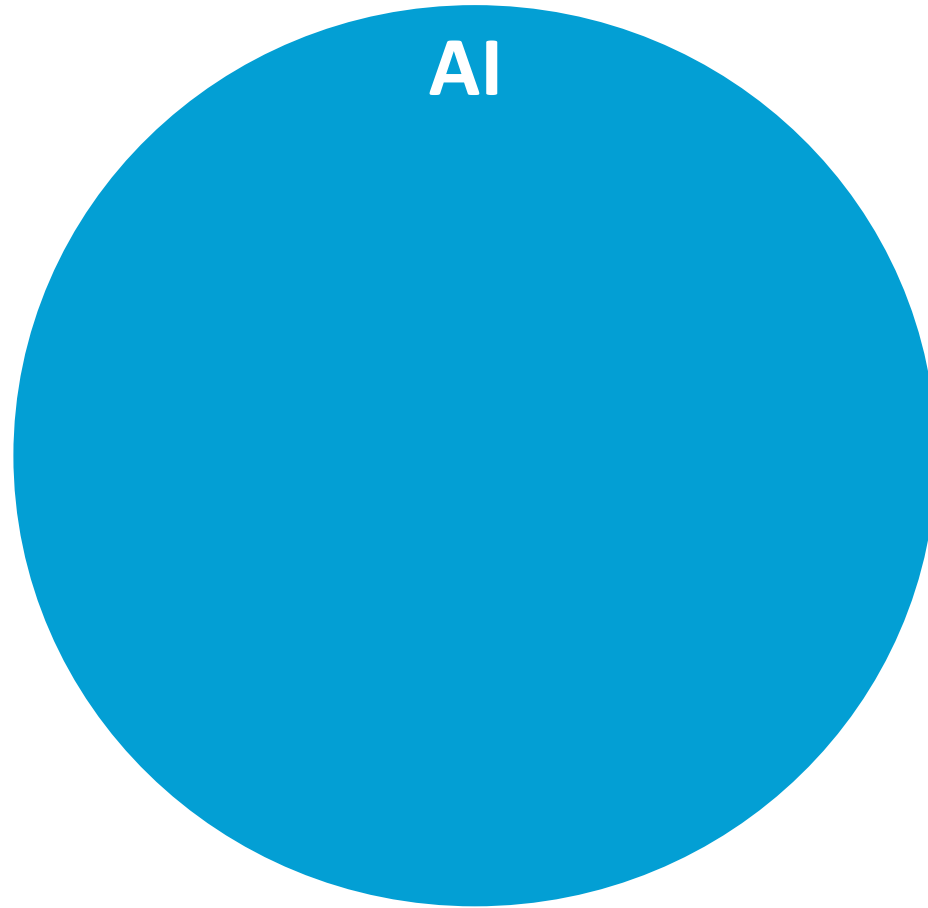
- Students can independently apply AI methods to aerospace problems for tasks such as regression and classification.
- Students can explain the theoretical properties, learning methods, and functioning of AI methods, with a focus on machine learning (ML) and deep neural networks (DNNs).



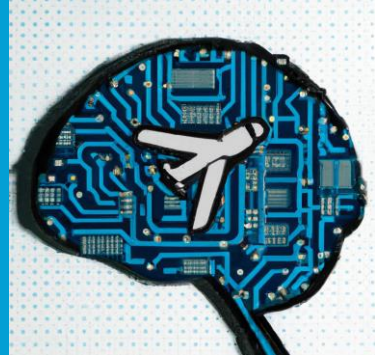
AI, machine learning, neural networks



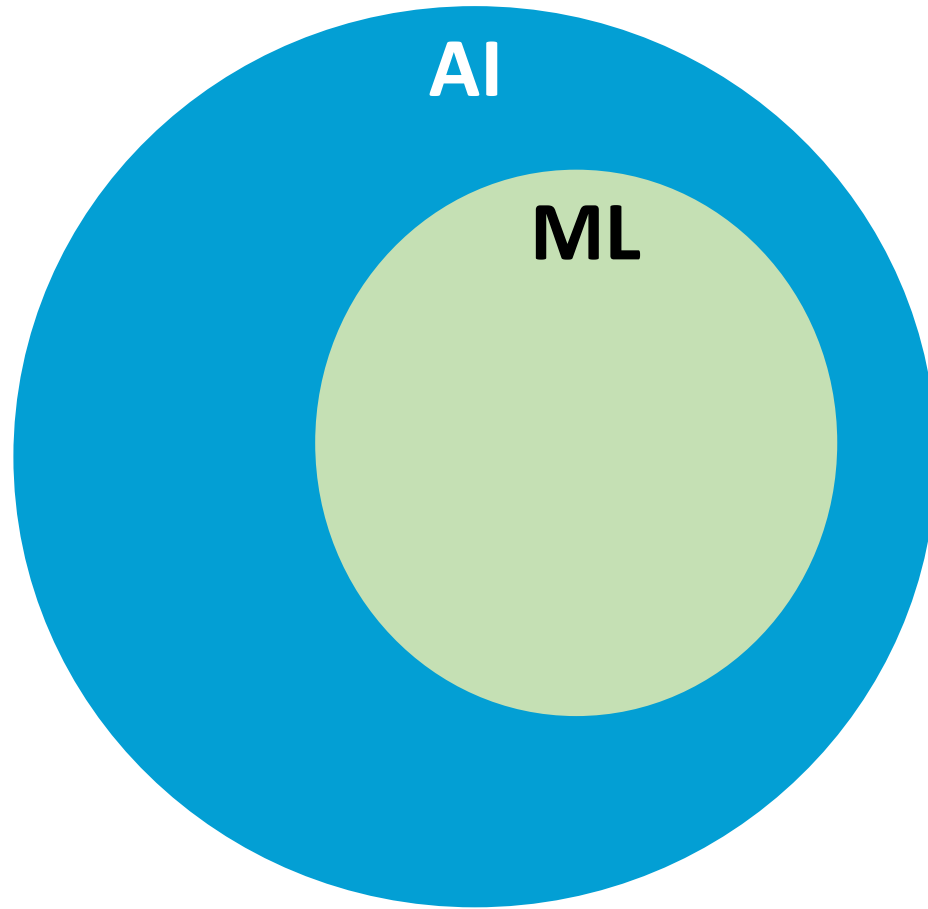
Venn diagram:



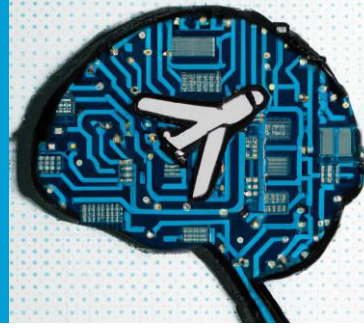
AI, machine learning, neural networks



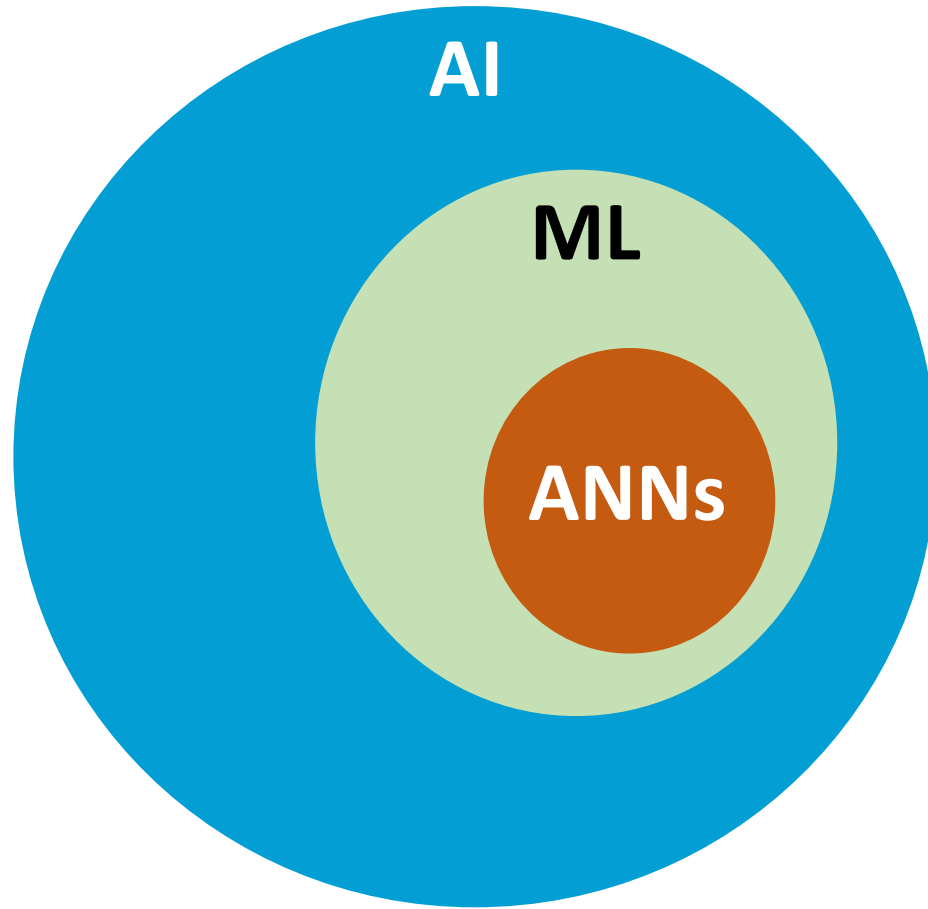
Venn diagram:



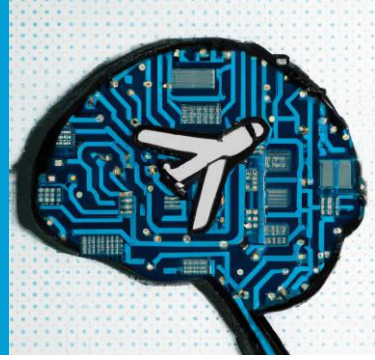
AI, machine learning, neural networks



Venn diagram:



Course setup

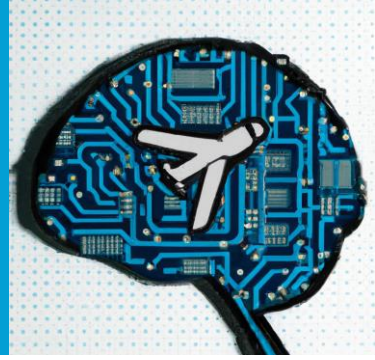


- 6 lectures and 8 practical sessions

Assessment

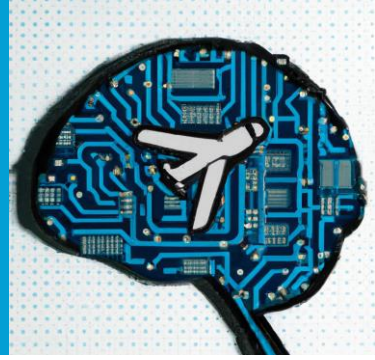
- Digital exam
 - answering questions on the theory
 - programming to solve machine learning problems
- A bonus point for the exam can be obtained by submitting a valid entry to the course's machine learning competition.

Topics and planning



1. Introduction to AI & ANNs
2. Deep neural networks
3. Generative AI
4. Recurrency
5. Physics-informed networks
6. Explainable AI

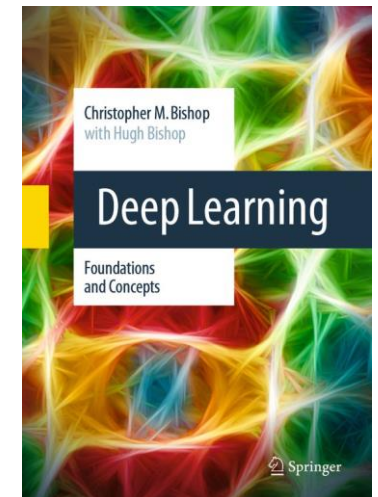
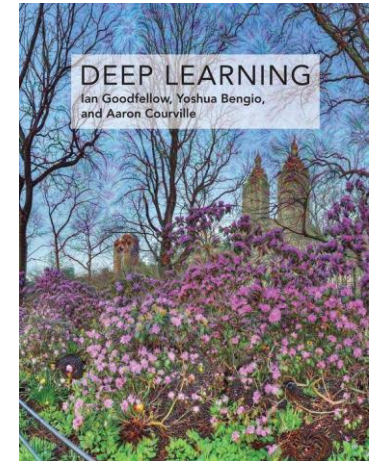
Reading material



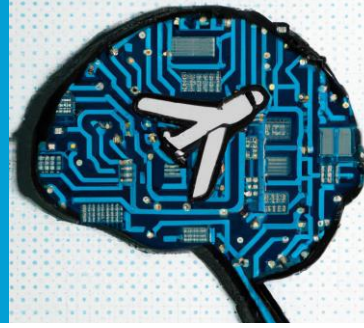
The lecture material is based on a variety of sources, among which two highly valued books:

“Deep learning”, by I. GoodFellow, Y. Bengio, and A. Courville (2016)

“Deep learning: Foundations and concepts”, by Bishop, C. M., & Bishop, H. (2023). Available via TU Delft [here](#).



Reading material



Interactive Python notebooks:

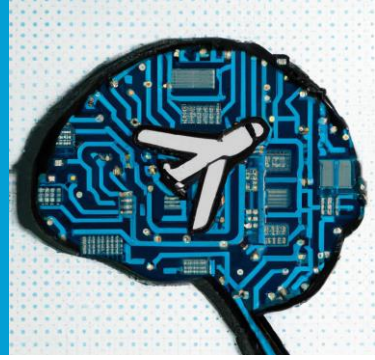
<https://github.com/guidoAI/ANNs>

<https://mybinder.org/v2/gh/guidoAI/ANNs/master>



We also refer to other sources, like other books, articles or online blogs / tutorials.

Competition

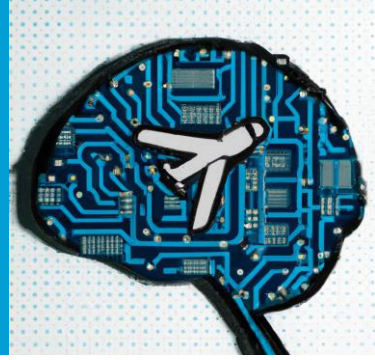


Students can obtain a bonus point by submitting a valid entry to the machine learning competition.

Valid entry =

- Performance higher than the baseline.
- 1-page report explaining the approach and main choices.
- Conversation with the TAs in which the student shows that they can motivate and explain the used neural network architecture, training process, etc.

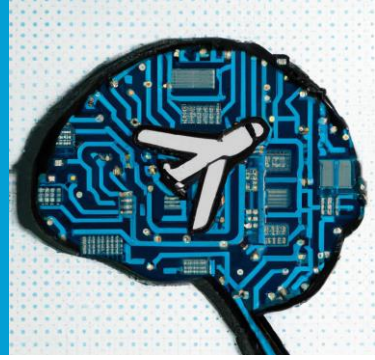
Skills



In AE4353 you will further hone your (Python) programming skills.

Raise your hand if:

Skills

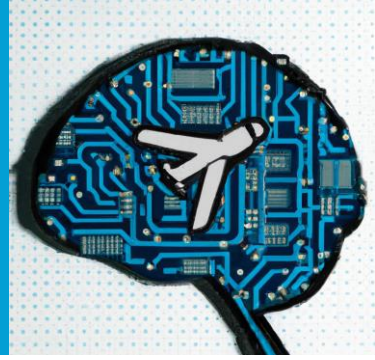


In AE4353 you will further hone your (Python) programming skills.

Raise your hand if:

1. You wonder why a snake 🐍 is relevant to programming.

Skills

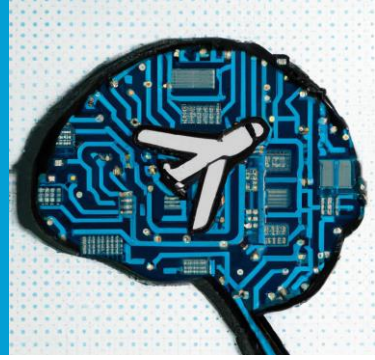


In AE4353 you will further hone your (Python) programming skills.

Raise your hand if:

1. You wonder why a snake 🐍 is relevant to programming.
2. You consider yourself still quite uncertain w.r.t. your Python skills.

Skills

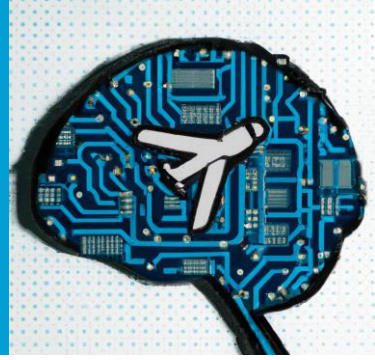


In AE4353 you will further hone your (Python) programming skills.

Raise your hand if:

1. You wonder why a snake 🐍 is relevant to programming.
2. You consider yourself still quite uncertain w.r.t. your Python skills.
3. You are confident that you'll make your way through.

Skills

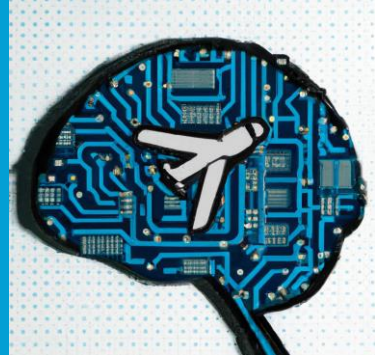


In AE4353 you will further hone your (Python) programming skills.

Raise your hand if:

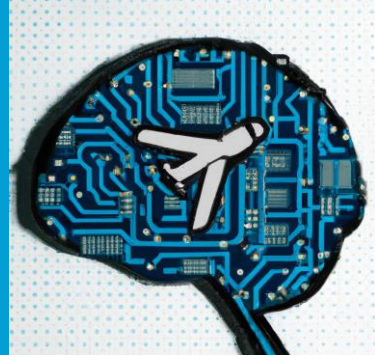
1. You wonder why a snake 🐍 is relevant to programming.
2. You consider yourself still quite uncertain w.r.t. your Python skills.
3. You are confident that you'll make your way through.
4. You consider yourself a Python expert 😎 **#list-comprehension**

Different levels



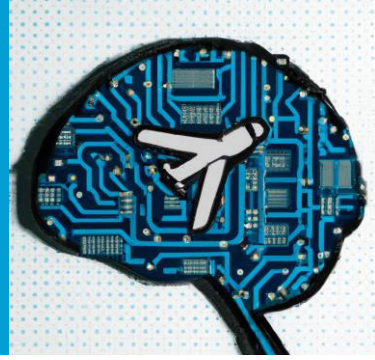
We try to accommodate different skill levels by having the standard notebooks and exercises, and providing **advanced topics** to those with more experience.

Coding & ChatGPT



You are free to use ChatGPT and tools like GitHub co-pilot during the course. They are excellent tools to help you learn how to program AI.

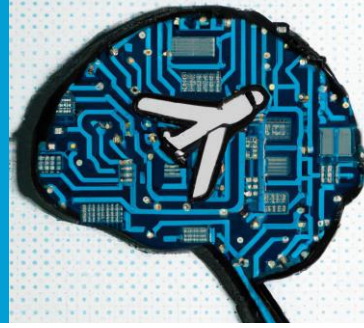
Coding & ChatGPT



You are free to use ChatGPT and tools like GitHub co-pilot during the course. They are excellent tools to help you learn how to program AI.

We advise to **not** use tools like Claude or Codex to do all the coding for you, because it hampers a thorough understanding of the methods.

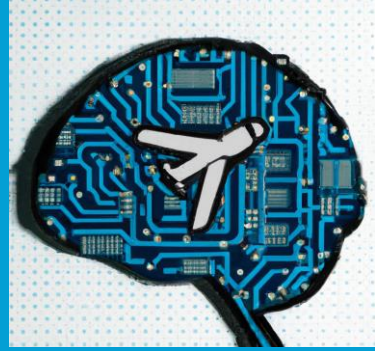
Coding & ChatGPT



You are free to use ChatGPT and tools like GitHub co-pilot during the course. They are excellent tools to help you learn how to program AI.

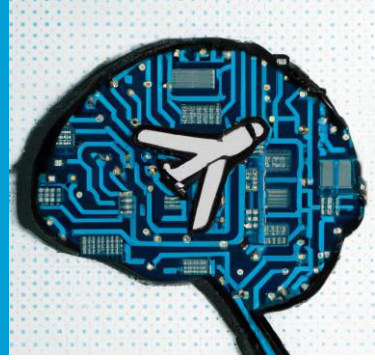
We advise to **not** use tools like Claude or Codex to do all the coding for you, because it hampers a thorough understanding of the methods.

Note: During the exam, these tools will not be available. You will have documentation on the relevant Python functions and classes, but for the rest you will be on your own.



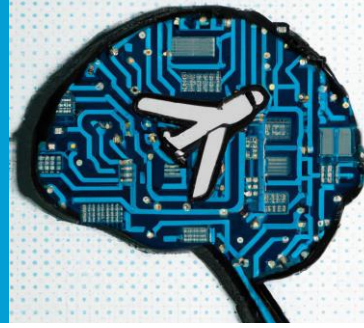
Machine learning fundamentals (I)

Machine learning tasks



- Classification
- Regression
- Anomaly detection
- Density estimation
- Synthesis
- Denoising
- ...

Main types of learning



Unsupervised learning

Given inputs $\mathbf{x}_i$ learn about the input structure / distribution: $P(\mathbf{x})$

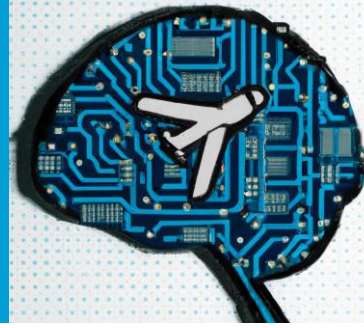
Supervised learning

Given input-target pairs $(\mathbf{x}_i, y_i^*)$ learn to predict y from $\mathbf{x}$: $P(y|\mathbf{x})$

Reinforcement learning

Learn a policy $\pi: f(\mathbf{x}) = \mathbf{y}$ to optimize a reward $r(\mathbf{x}, \mathbf{y})$

Main types of learning



Unsupervised learning

Given inputs $\mathbf{x}_i$ learn about the input structure / distribution: $P(\mathbf{x})$

Supervised learning

Given input-target pairs $(\mathbf{x}_i, y_i^*)$ learn to predict y from $\mathbf{x}$: $P(y|\mathbf{x})$

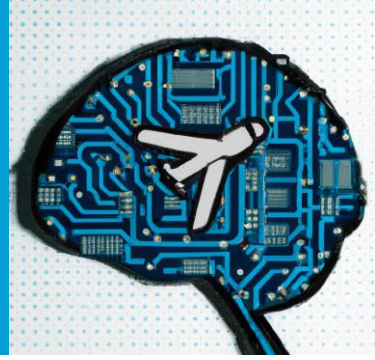
Reinforcement learning

Learn a policy $\pi: f(\mathbf{x}) = \mathbf{y}$ to optimize a reward $r(\mathbf{x}, \mathbf{y})$

Note:

Reinforcement learning is taught in AE4350, in the course on bio-inspired AI for aerospace.

Supervised learning



Each input vector x_i is also called a “**sample**”.

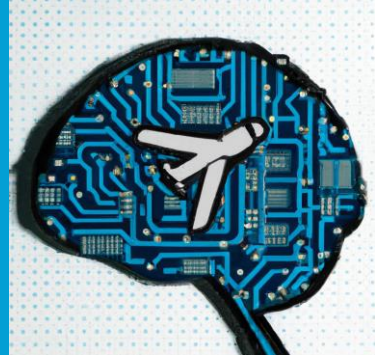
Elements j of the vector, $x_{i,j}$, are called “**features**”.

Sometimes all input data is stored in a single matrix, the “**design matrix**”:

$$X \in \mathbb{R}^{m \times n}$$

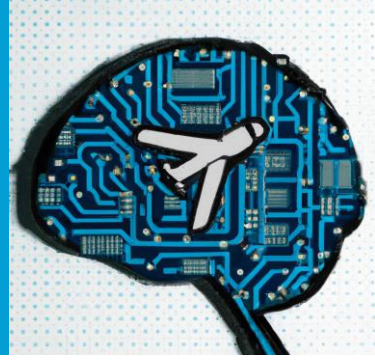
Where m is the number of samples and n the number of features.

The challenge



Supervised learning assumes that:

The challenge

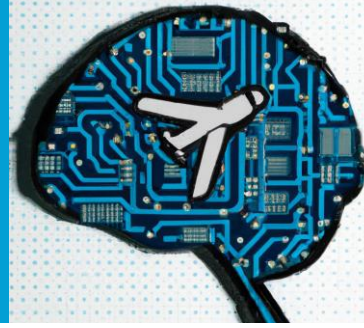


Supervised learning assumes that:

- The samples are **independent** from each other:

$$\forall i, j: P(\mathbf{x}_i | \mathbf{x}_j) = P(\mathbf{x}_i)$$

The challenge



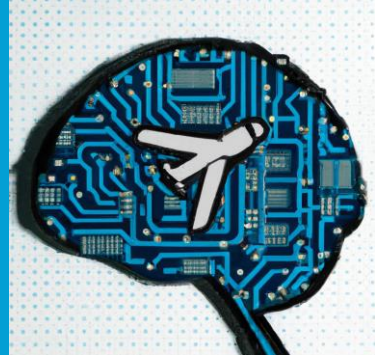
Supervised learning assumes that:

- The samples are **independent** from each other:

$$\forall i, j: P(\mathbf{x}_i | \mathbf{x}_j) = P(\mathbf{x}_i)$$

- The training and test input samples are **identically distributed**, i.e., they come from the same probability distribution $P(\mathbf{x}) = P_{\text{train}}(\mathbf{x}) = P_{\text{test}}(\mathbf{x})$.

The challenge



Supervised learning assumes that:

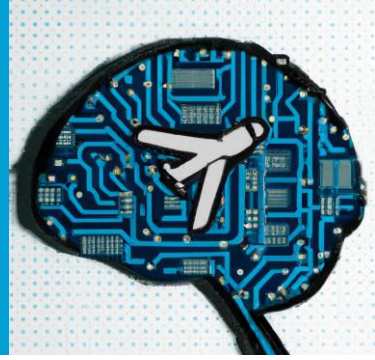
- The samples are **independent** from each other:

$$\forall i, j: P(\mathbf{x}_i | \mathbf{x}_j) = P(\mathbf{x}_i)$$

- The training and test input samples are **identically distributed**, i.e., they come from the same probability distribution $P(\mathbf{x}) = P_{\text{train}}(\mathbf{x}) = P_{\text{test}}(\mathbf{x})$.

These assumptions are called the “**i.i.d. assumptions**”.

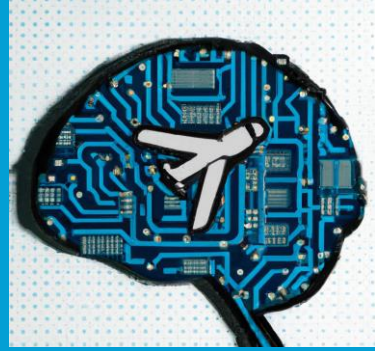
The challenge



In reality, these assumptions are often violated.

The most obvious problem is that the training set can be quite different from the test set.

A model $f(\mathbf{x}) = y$ that fits the training data very well, may behave poorly on the test set.



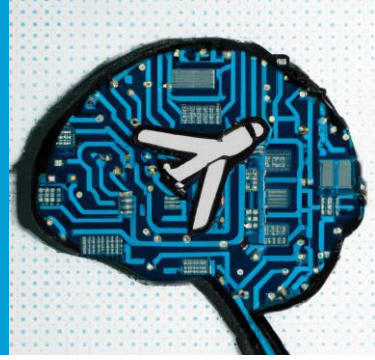
Artificial Neural Networks

Artificial neural networks

Loosely inspired by the brain, artificial neural networks are one of the most important “tools” for achieving artificial intelligence.



In general used to learn functions

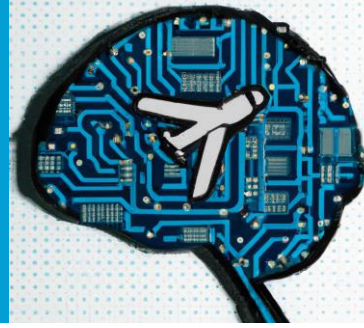


Input
numbers



Output
numbers

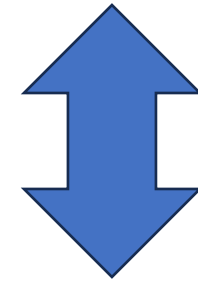
In general used to learn functions



Input
numbers



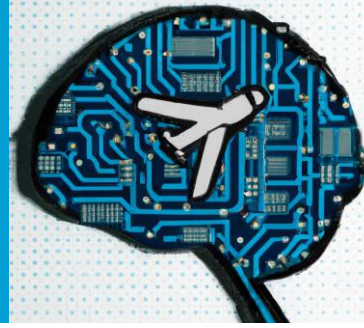
Output
numbers



Target
numbers

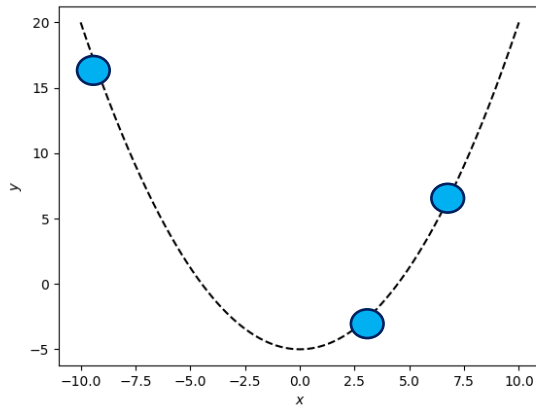
Supervised learning

In general used to learn functions

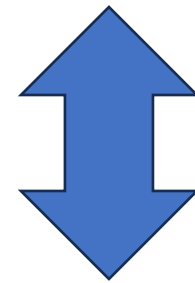


Input numbers

Output numbers

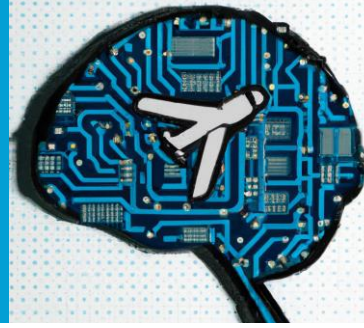


$x =$ 3.54453943
6.73564341
-9.25389741
...



$y =$ -3.18839945
5.90074899
15.15246902
...

In general used to learn functions



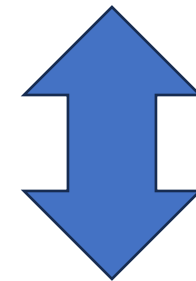
Input
numbers



Output
numbers



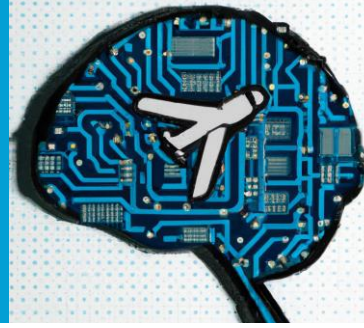
(250, 200, 240), (255, 250, 244), ...
(251, 220, 233), ...



Cat: (0,1)

Dog: (1,0)

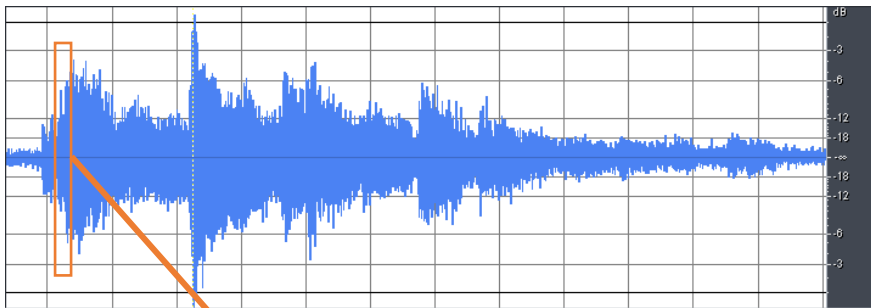
In general used to learn functions



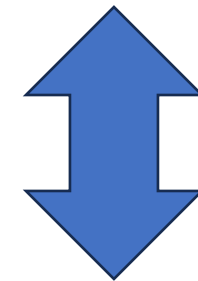
Input
numbers



Output
numbers



..., -12, -11.8, -10.2, -8.2, ...

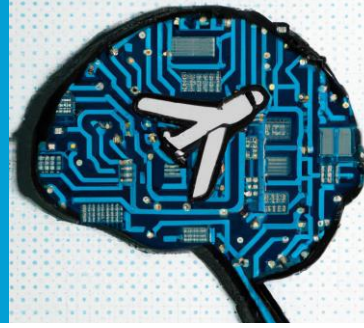


$A = (1, 0, 0, \dots)$

$B = (0, 1, 0, \dots)$

...

In general used to learn functions



x

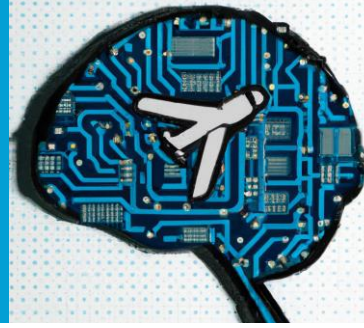


y



t

In general used to learn functions



x



y

Dataset:

$\langle x_1, t_1 \rangle$

$\langle x_2, t_2 \rangle$

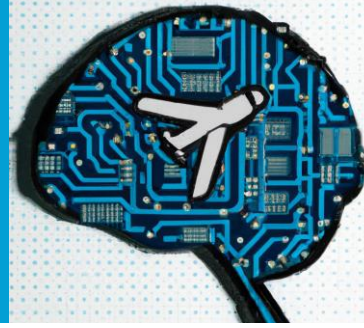
$\langle x_3, t_3 \rangle$

...



t

In general used to learn functions



x



$\hat{y}$

Dataset:

$\langle x_1, y_1^* \rangle$

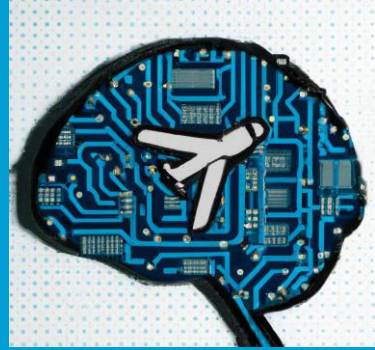
$\langle x_2, y_2^* \rangle$

$\langle x_3, y_3^* \rangle$

...

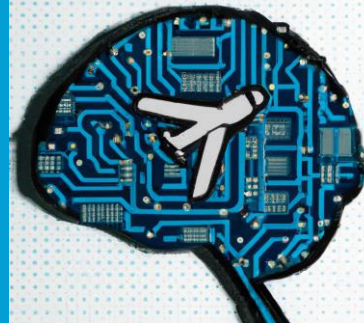


y^*



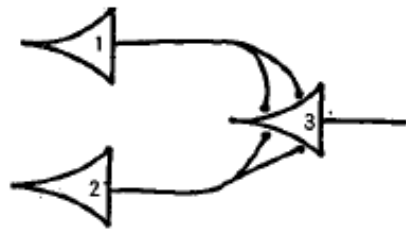
The Perceptron

Perceptron



Based on neurological findings of the time, McCulloch & Pitts (1943) proposed the *perceptron*.

The perceptron had a fixed structure that could implement logical functions.

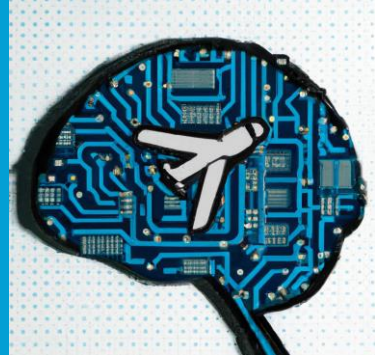


BULLETIN OF
MATHEMATICAL BIOPHYSICS
VOLUME 5, 1943

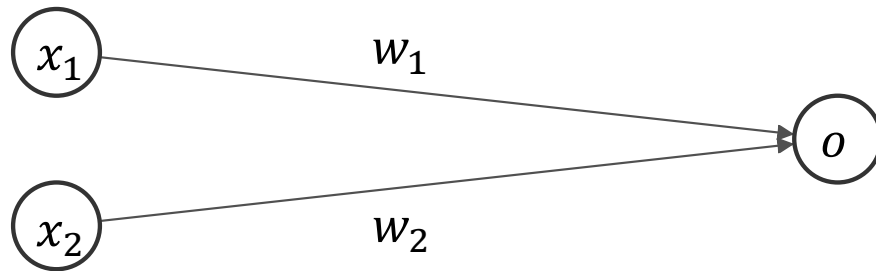
A LOGICAL CALCULUS OF THE
IDEAS IMMANENT IN NERVOUS ACTIVITY

WARREN S. MCCULLOCH AND WALTER PITTS

Perceptron



In 1958 Frank Rosenblatt introduced the perceptron as we know it today:

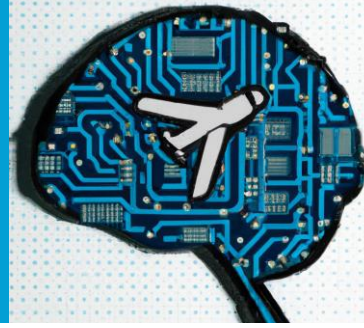


Input Layer $\in \mathbb{R}^2$

Output Layer $\in \mathbb{R}^1$

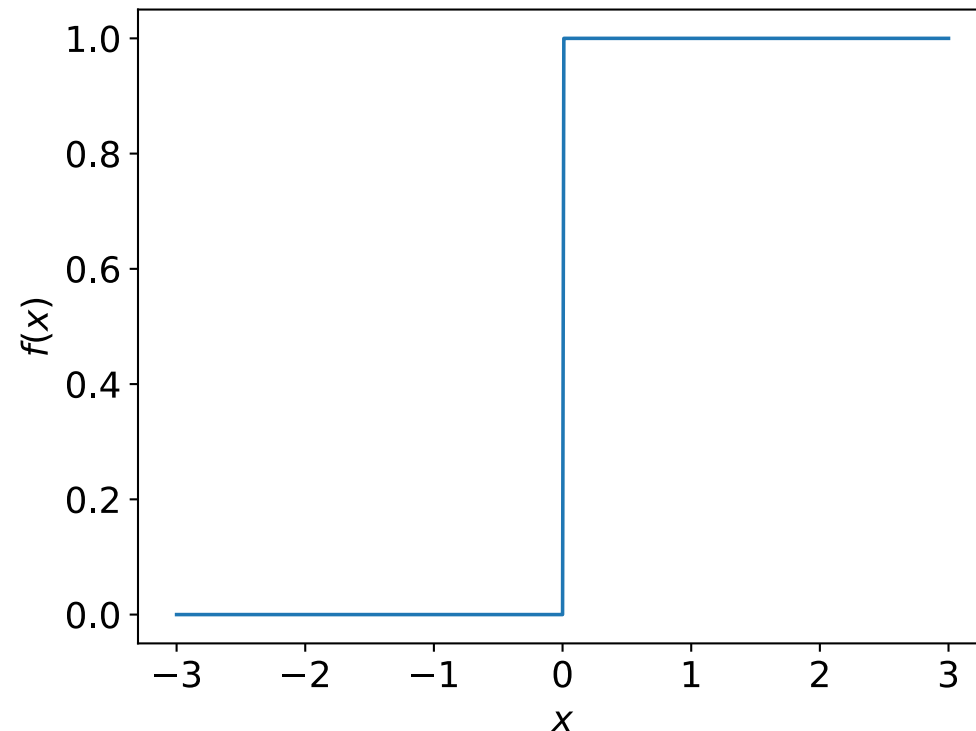
$$o = f\left(\sum_i w_i x_i\right)$$

Perceptron

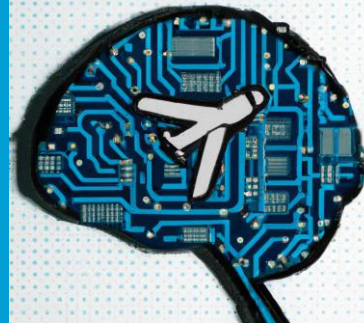


The weighted inputs are summed and served as input to the activation function – initially a step function.

$$o = f\left(\sum_i w_i x_i\right)$$

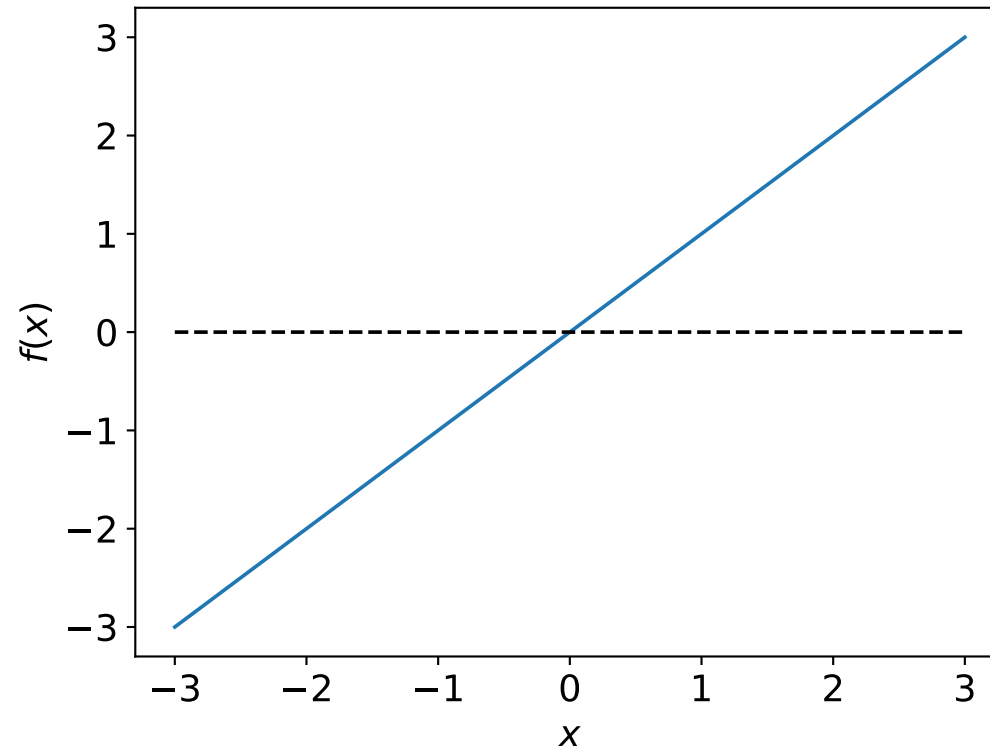


Perceptron

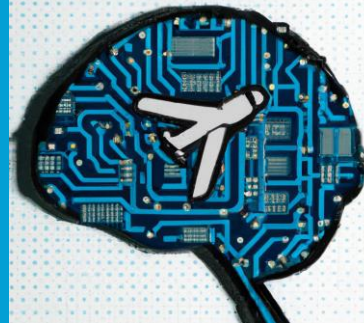


It can be used for classification (with the step activation function) or for regression (with, e.g., the linear activation function).

$$o = f\left(\sum_i w_i x_i\right)$$

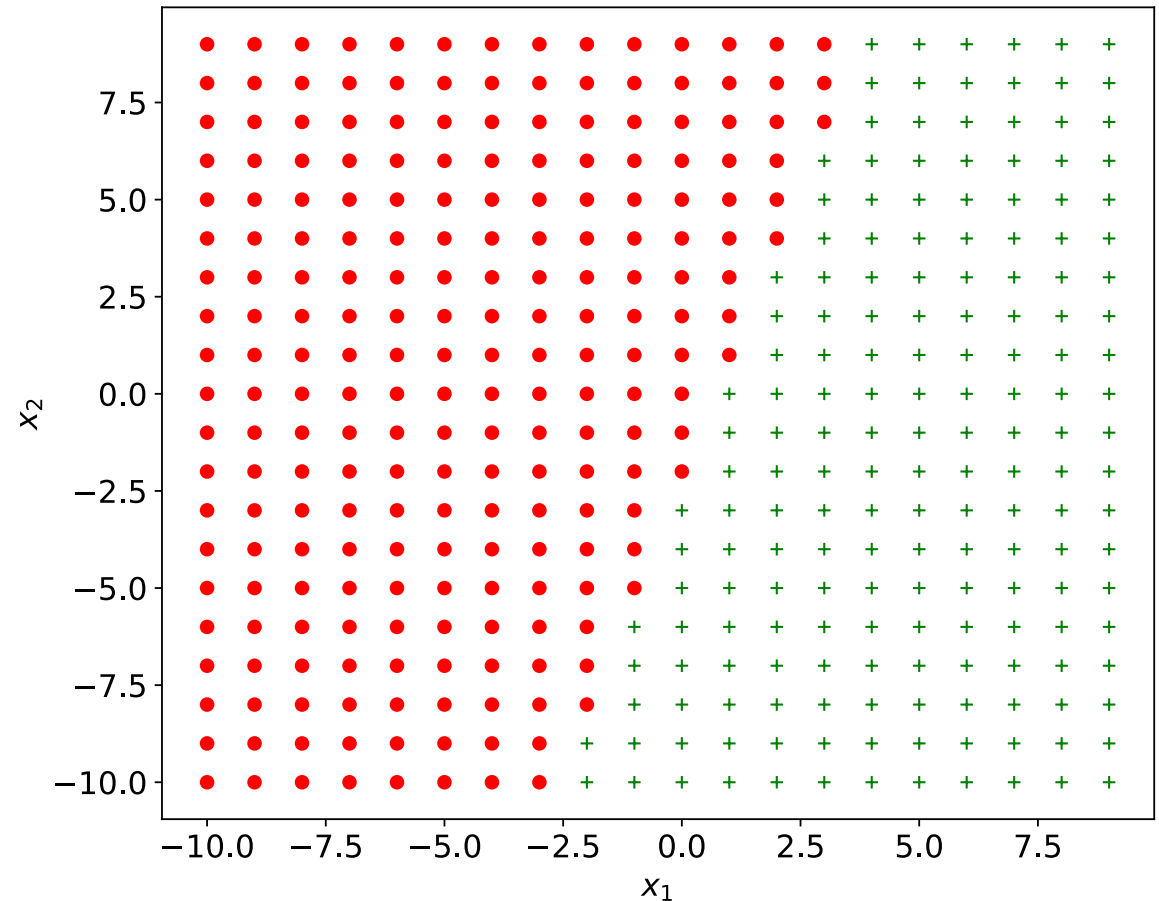


Perceptron

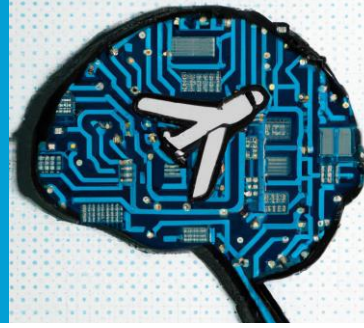


Example of a classification problem that can be solved with a perceptron.

- Negative
- + Positive

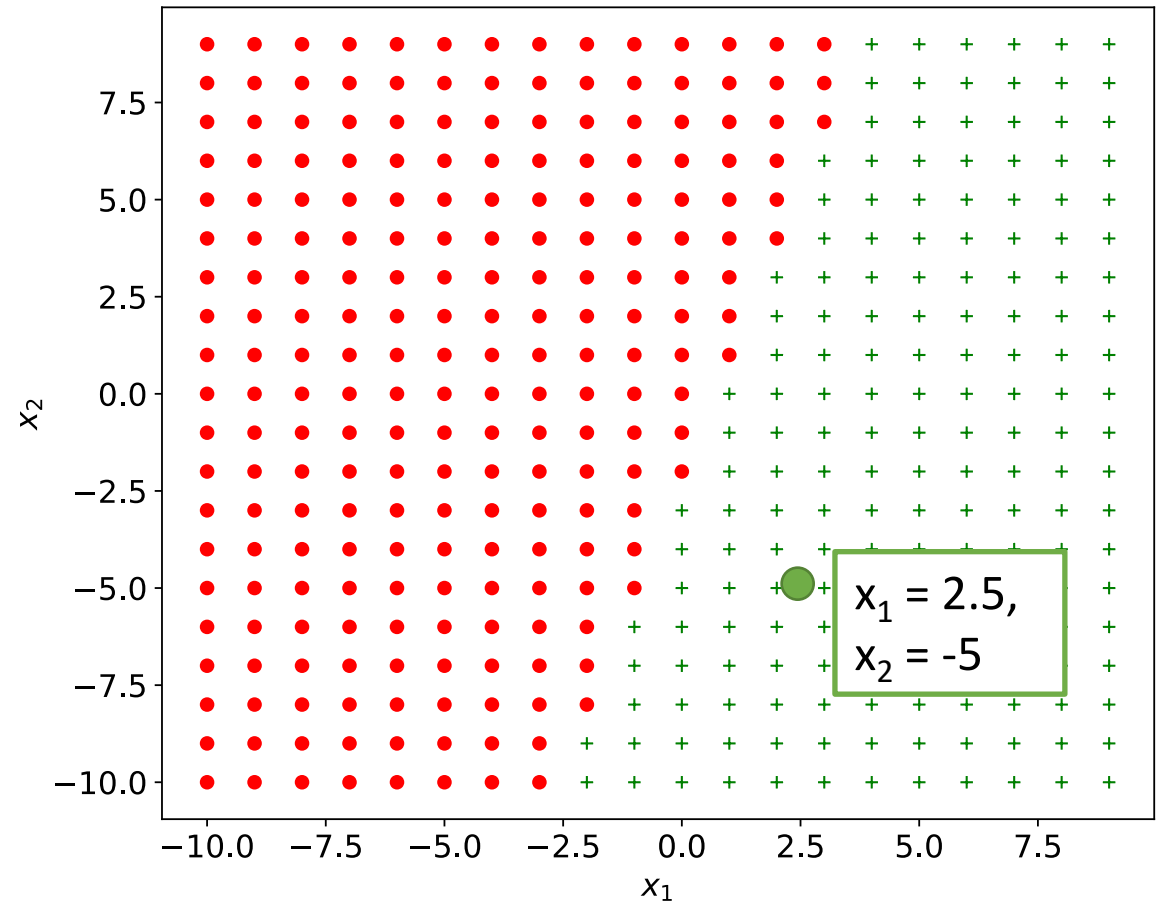


Perceptron

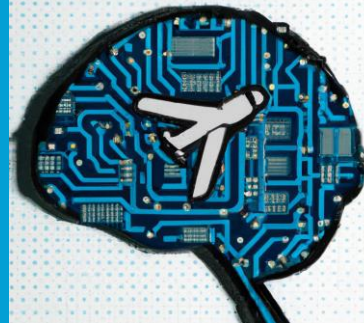


Example of a classification problem that can be solved with a perceptron.

- Negative
- + Positive

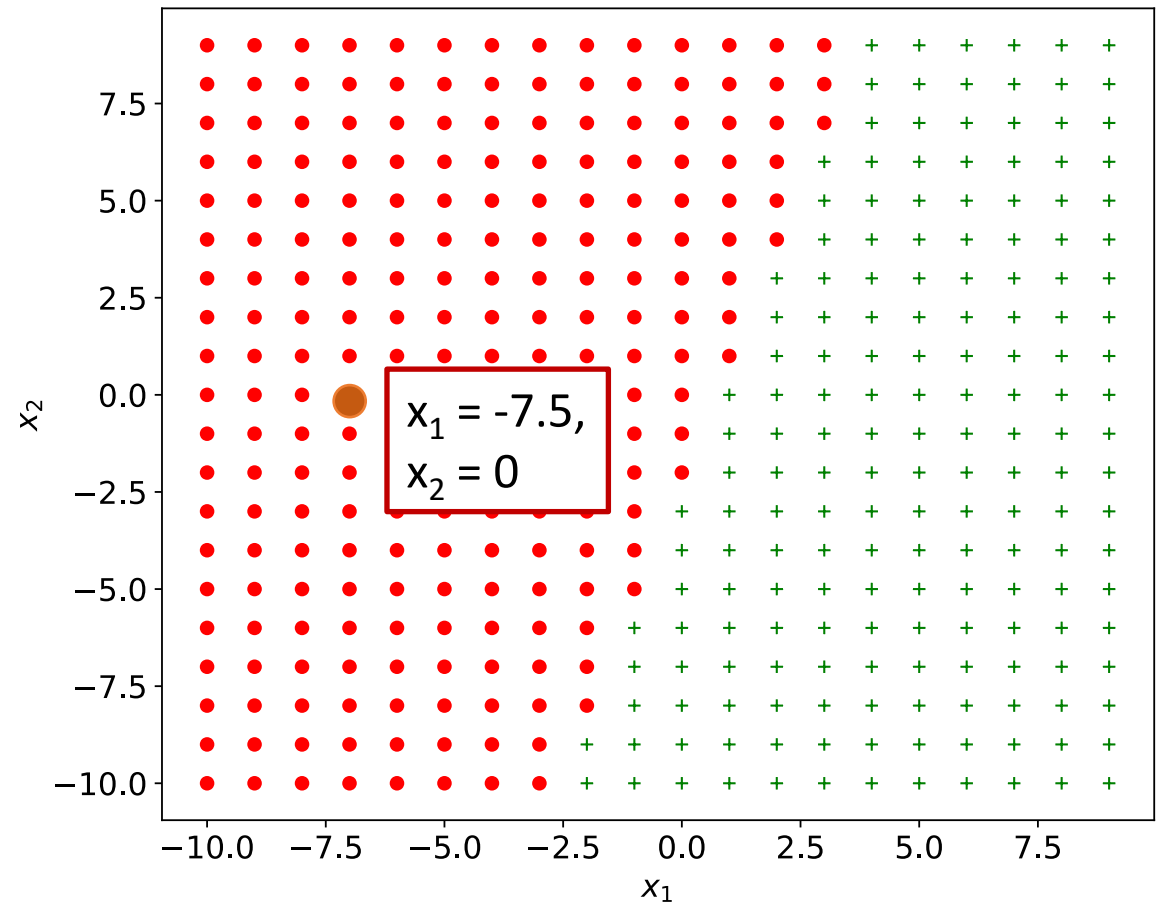


Perceptron

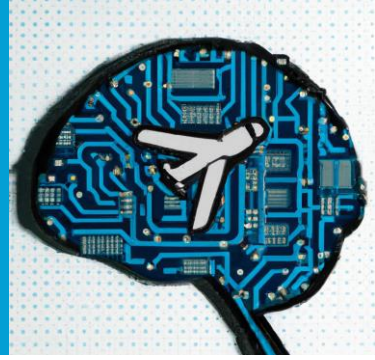


Example of a classification problem that can be solved with a perceptron.

- Negative
- + Positive

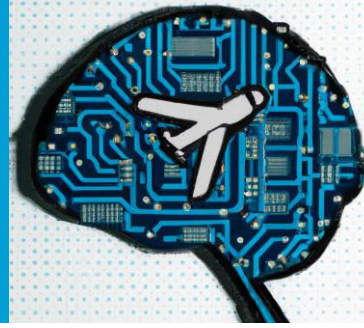


Learning the weights



Given a list of inputs (x_1, x_2) and corresponding targets t , can you think of a way how to learn the weights (w_1, w_2) ?

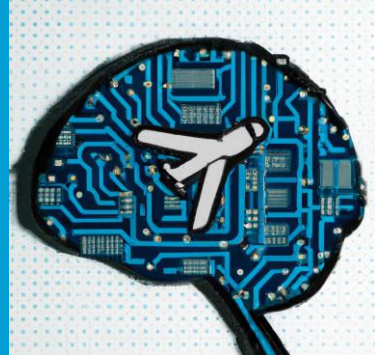
Learning the weights



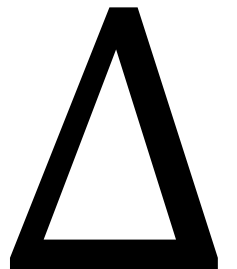
Given a list of inputs (x_1, x_2) and corresponding targets t , can you think of a way how to learn the weights (w_1, w_2) ?

For example, what to do if we have $w_1 = 1, w_2 = 1$, and we receive $x_1 = 0, x_2 = 3$ with $t = 0$?

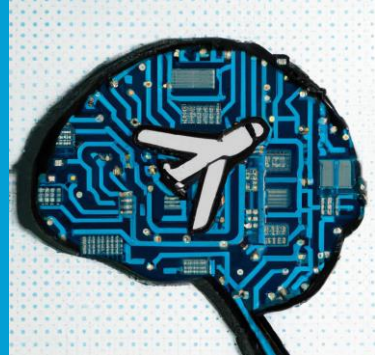
The Delta learning rule



The “Delta rule” is a perceptron learning rule introduced by Widrow and Hoff in 1962. The idea is to use a **cost function** and perform **gradient descent** on the parameters of the neural network in order to reduce the cost over time.



The Delta learning rule

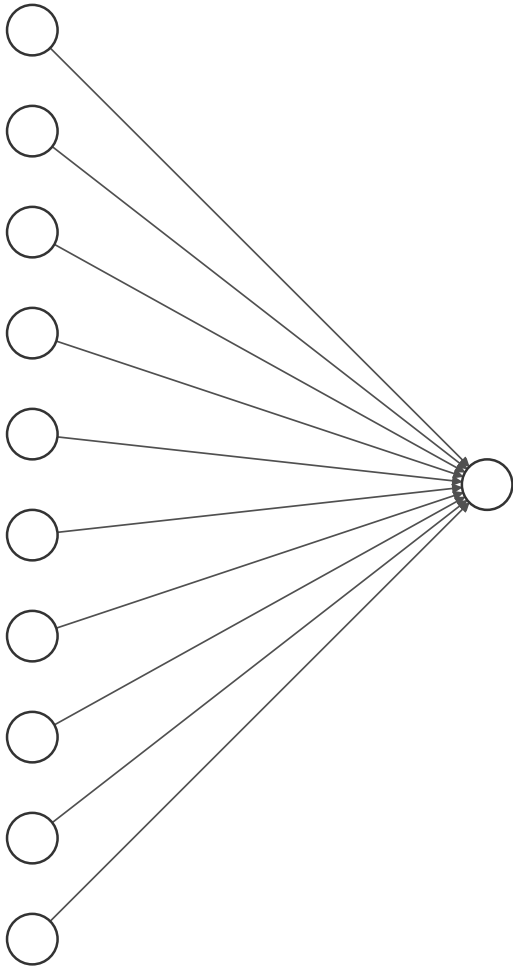
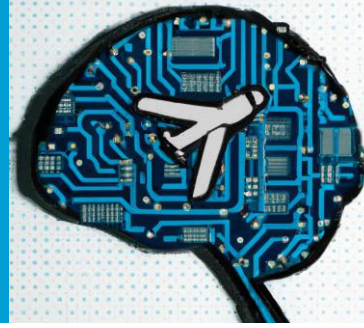


A popular cost function is the quadratic cost function:

$$e = (o - t)^2$$

, where e is the error, t is the target, and o is the network's output.

The Delta learning rule



Input Layer $\in \mathbb{R}^{10}$

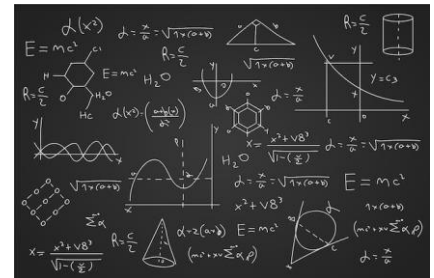
Output Layer $\in \mathbb{R}^1$

$$e = (o - t)^2$$

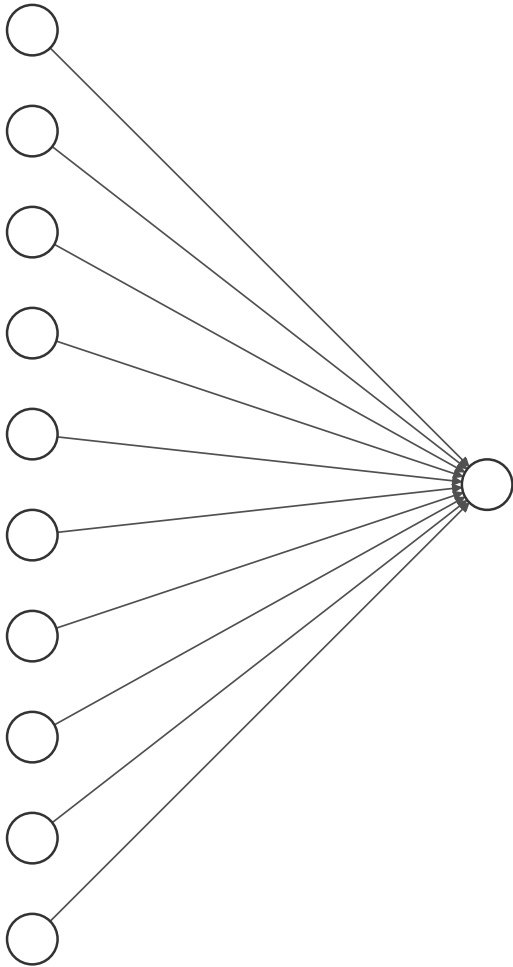
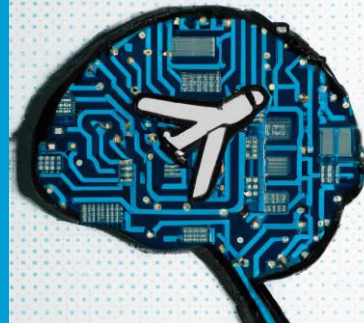
$$o = f(a) = f\left(\sum_i w_i x_i\right)$$

$$\frac{\partial e}{\partial w_i} = ?$$

$$\Delta w_i = -\alpha \frac{\partial e}{\partial w_i}$$



The Delta learning rule

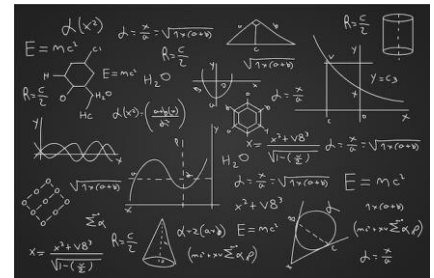


$$e = (o - t)^2$$

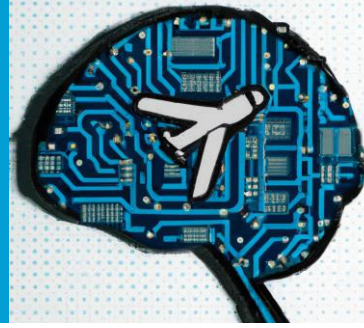
$$o = f(a) = f\left(\sum_i w_i x_i\right)$$

$$\frac{\partial e}{\partial w_i} = 2(o - t)f'(a)x_i$$

$$\Delta w_i = -\alpha \frac{\partial e}{\partial w_i}$$



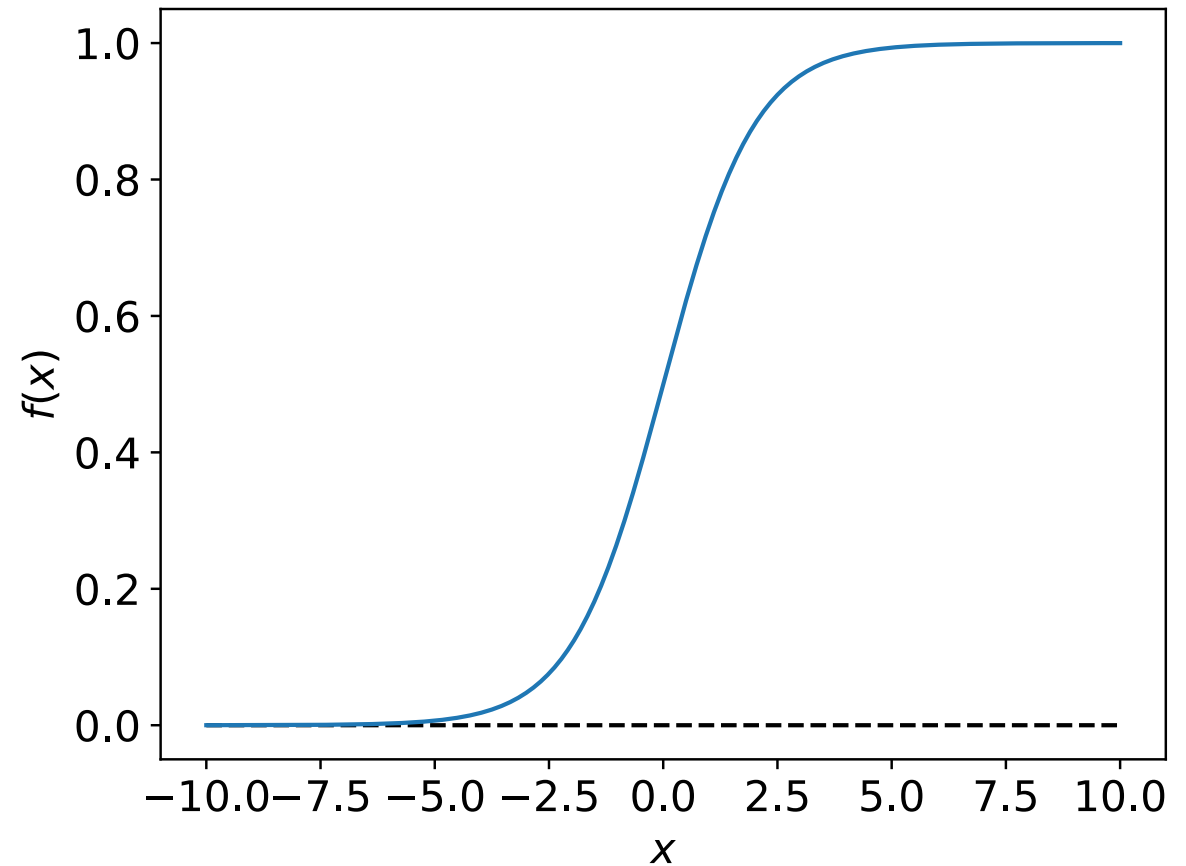
Sigmoid activation function



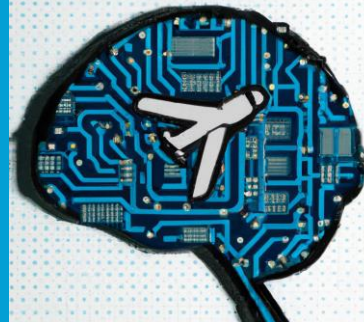
The Delta rule requires a differentiable activation function.

A popular such function is the sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$



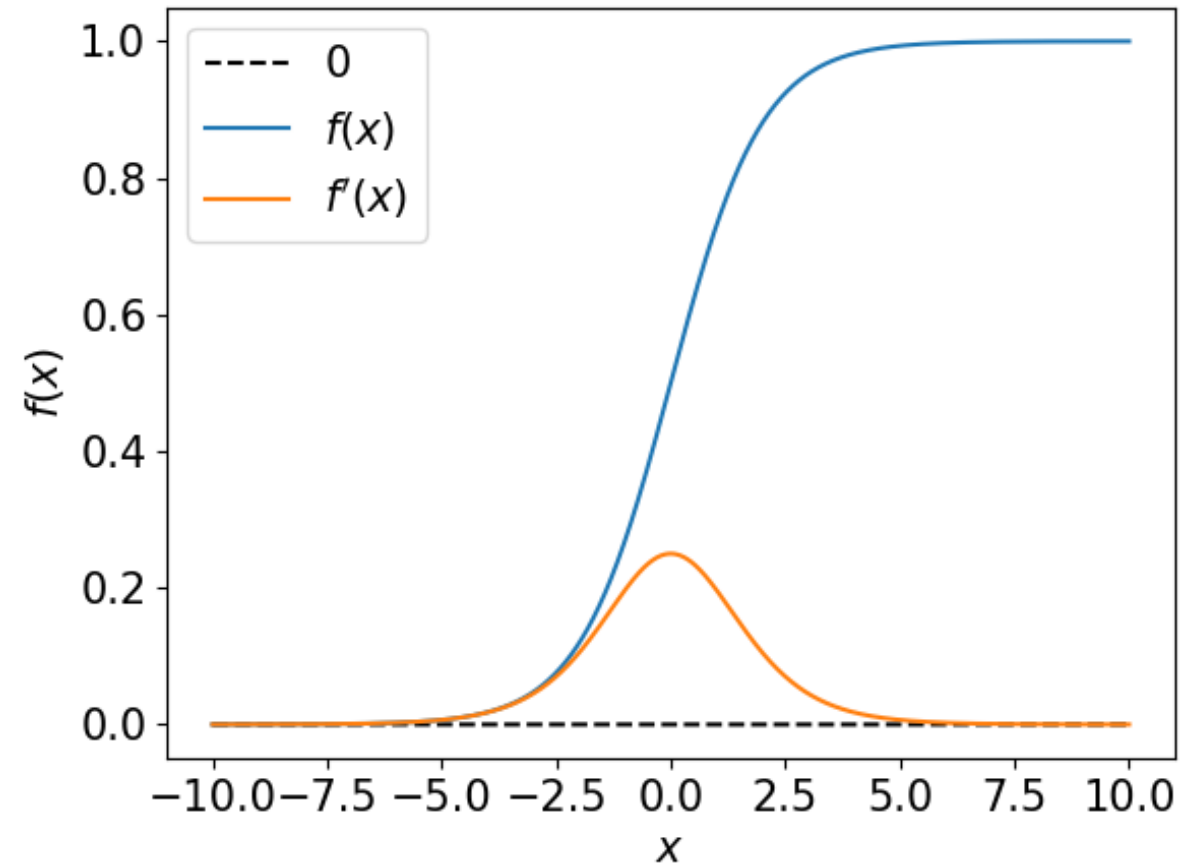
Sigmoid activation function



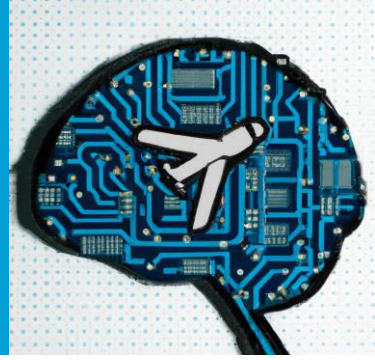
The Delta rule requires a differentiable activation function.

A popular such function is the sigmoid function:

$$f(x) = \frac{1}{1 + e^{-x}}$$



Derivative of the sigmoid function

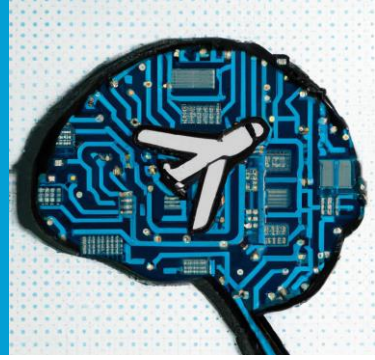


The derivative of the sigmoid function can be expressed as:

$$f'(x) = f(x)(1 - f(x))$$



Sigmoid activation function



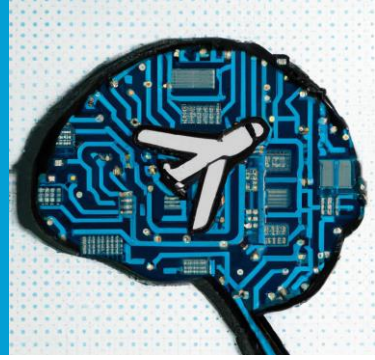
Advantages:

- ...
- ...

Disadvantage:

- ...

Sigmoid activation function



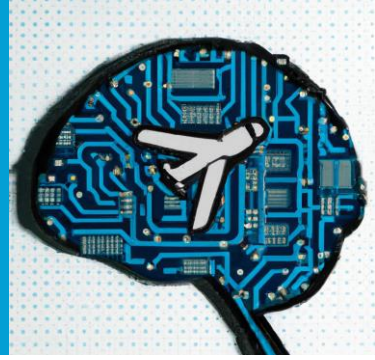
Advantages:

- It can approximate a step function
- It is bounded to $[0,1]$

Disadvantage:

- The derivative becomes very close to zero when saturated

(Naïve) learning setup



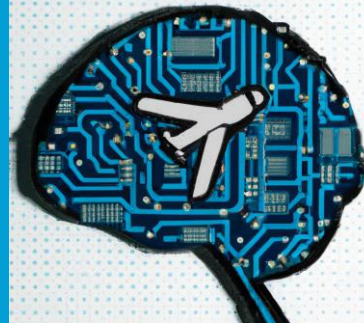
Require: Training data set with N samples:

$$\langle \mathbf{x}, t \rangle_j \text{ for } j = 1:N$$

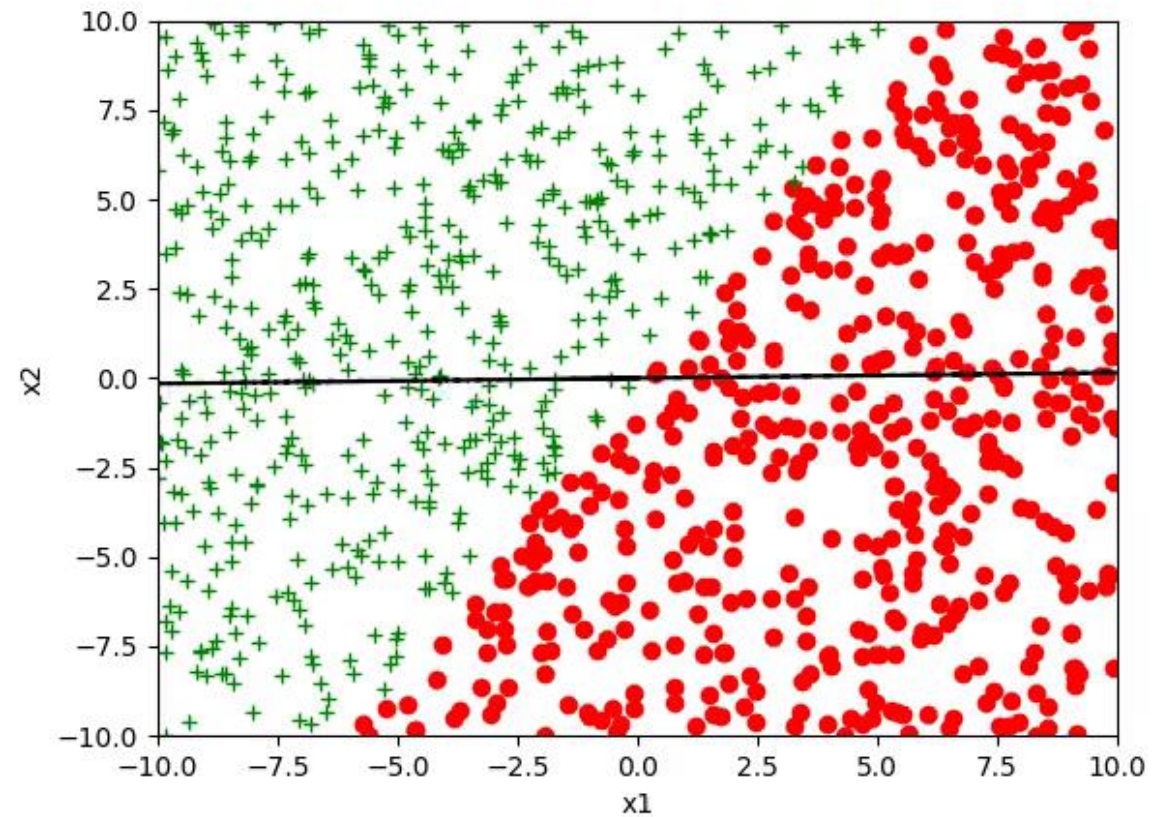
For a number of iterations I , do:

- Loop over all samples in the training set:
 - Calculate the output o for each sample $\mathbf{x}$
 - Calculate and apply a delta for each weight w_i

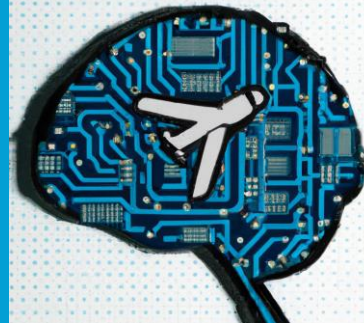
Naïve learning scheme



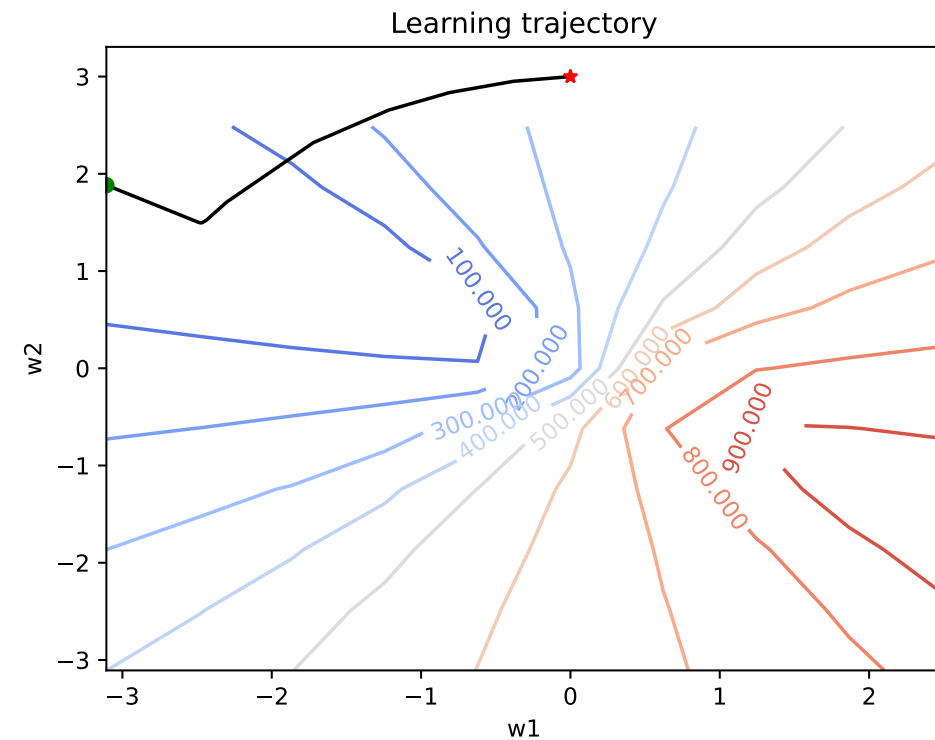
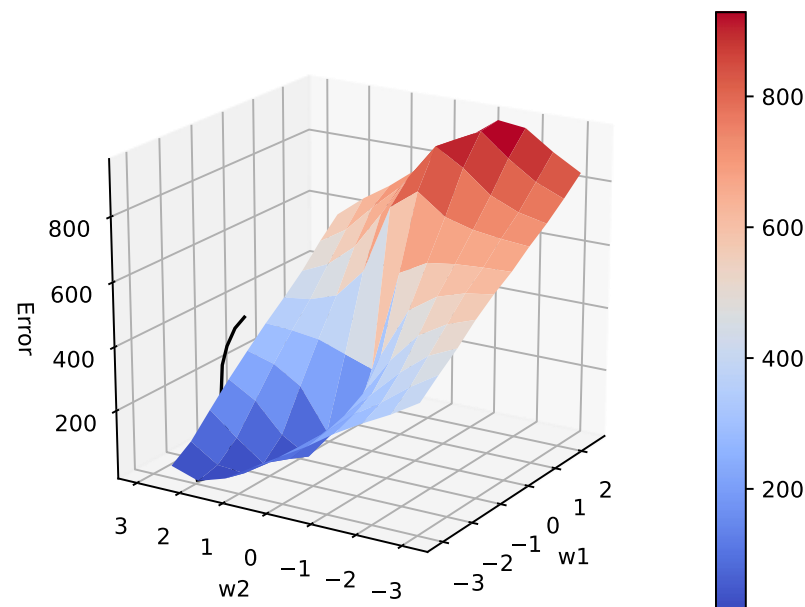
Learning progress during a single iteration:



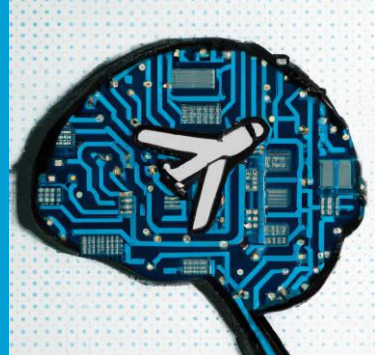
Learning as parameter optimization



Trajectory of the delta learning rule on the error surface:



Limits of the perceptron



In 1969 Minsky and Papert highlighted a major limit of perceptrons:

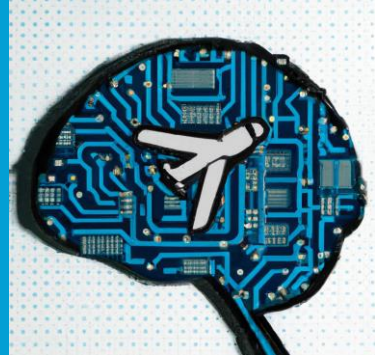
Perceptrons can only learn linearly separable functions!

This led to the first AI winter... 🤖

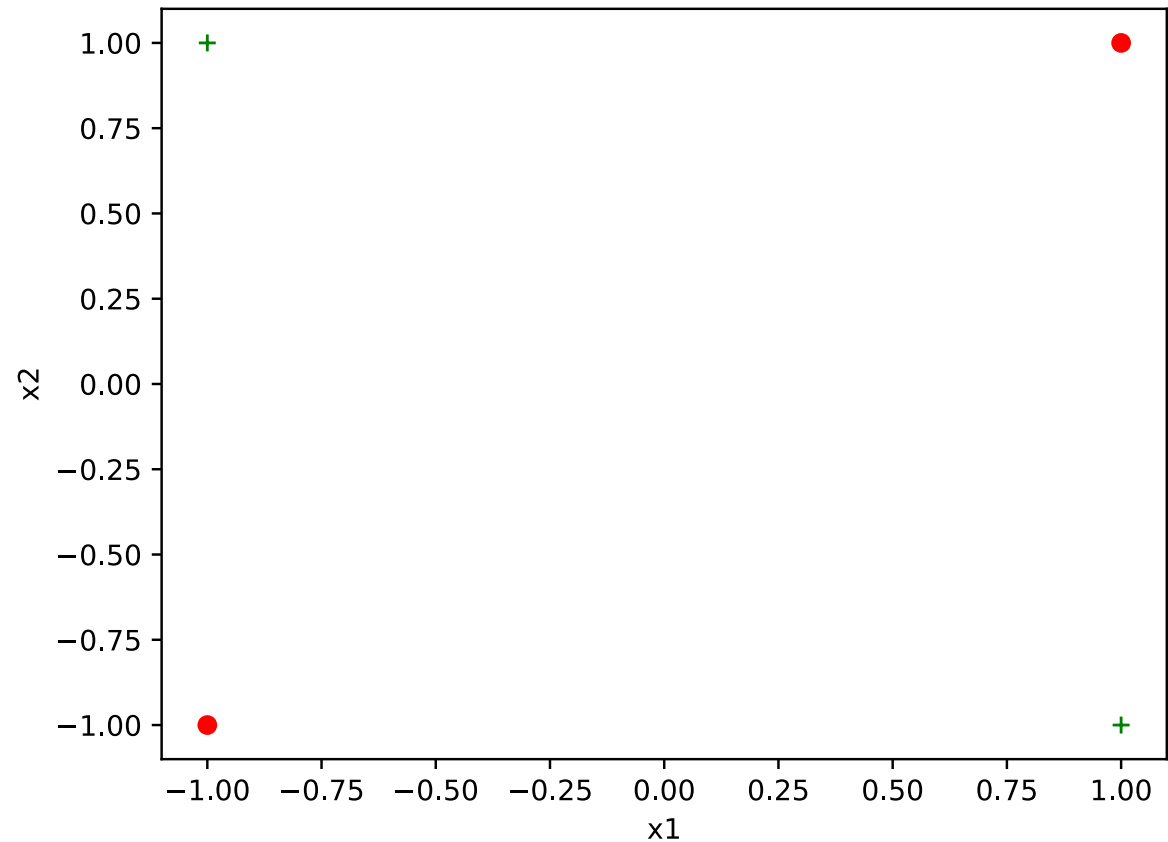


Drawing of AI winter by [Dall-E](#).

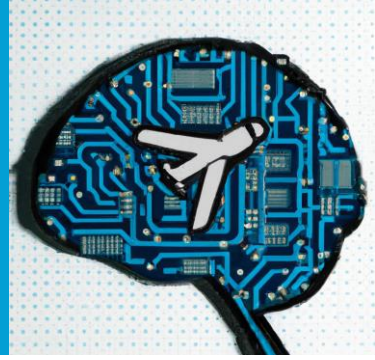
Limits of the perceptron



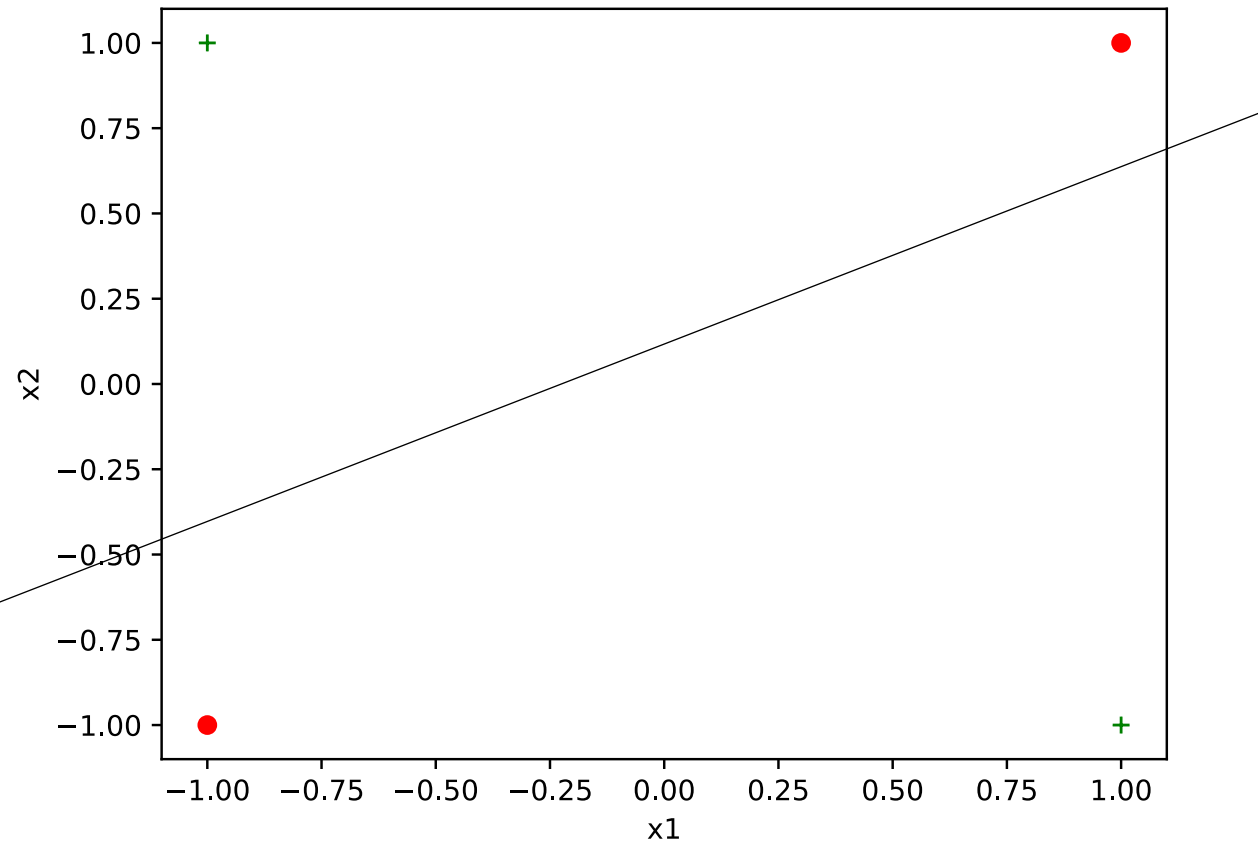
The XOR problem:



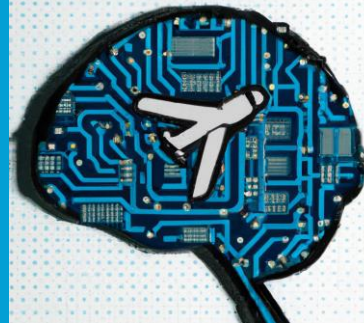
Limits of the perceptron



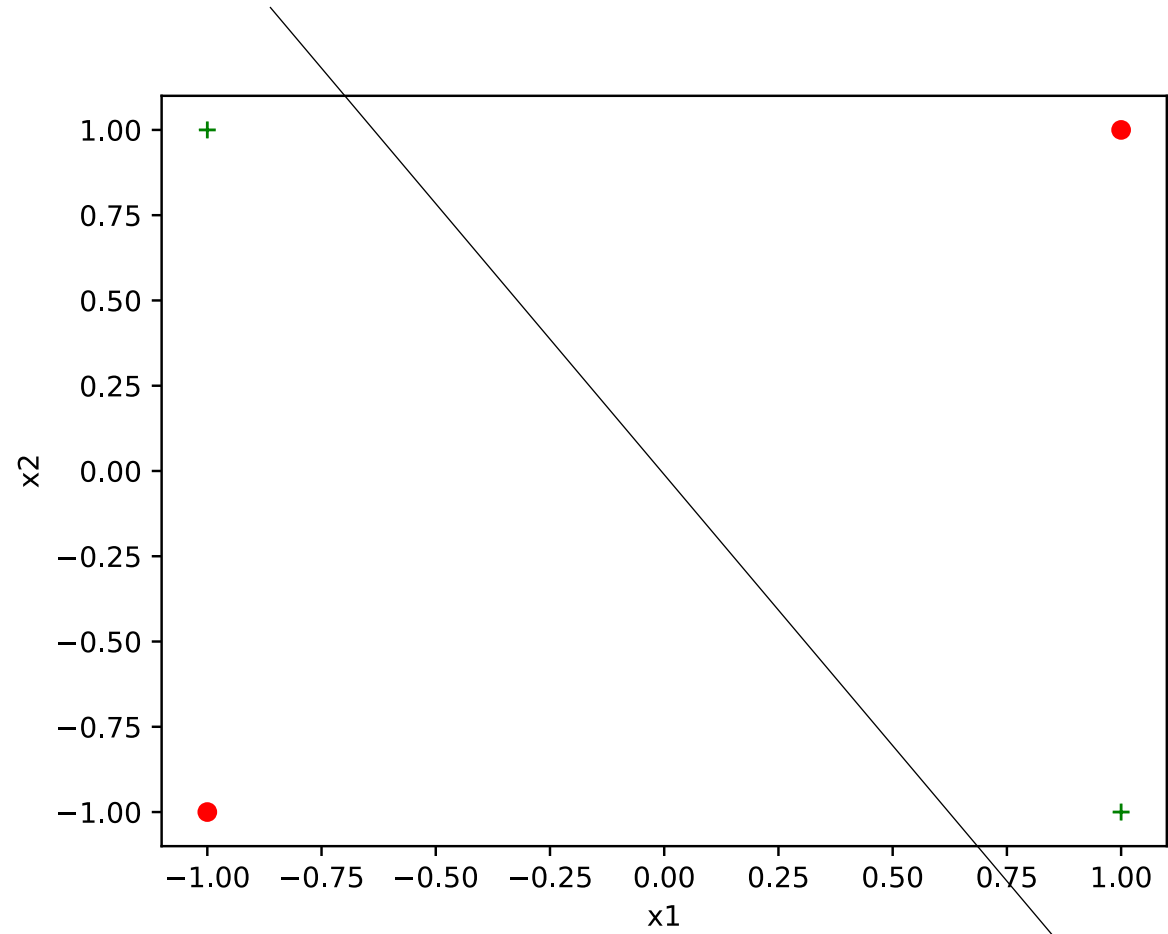
The XOR problem:



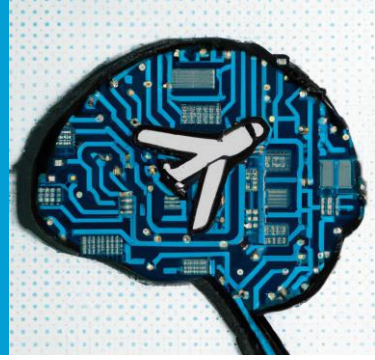
Limits of the perceptron



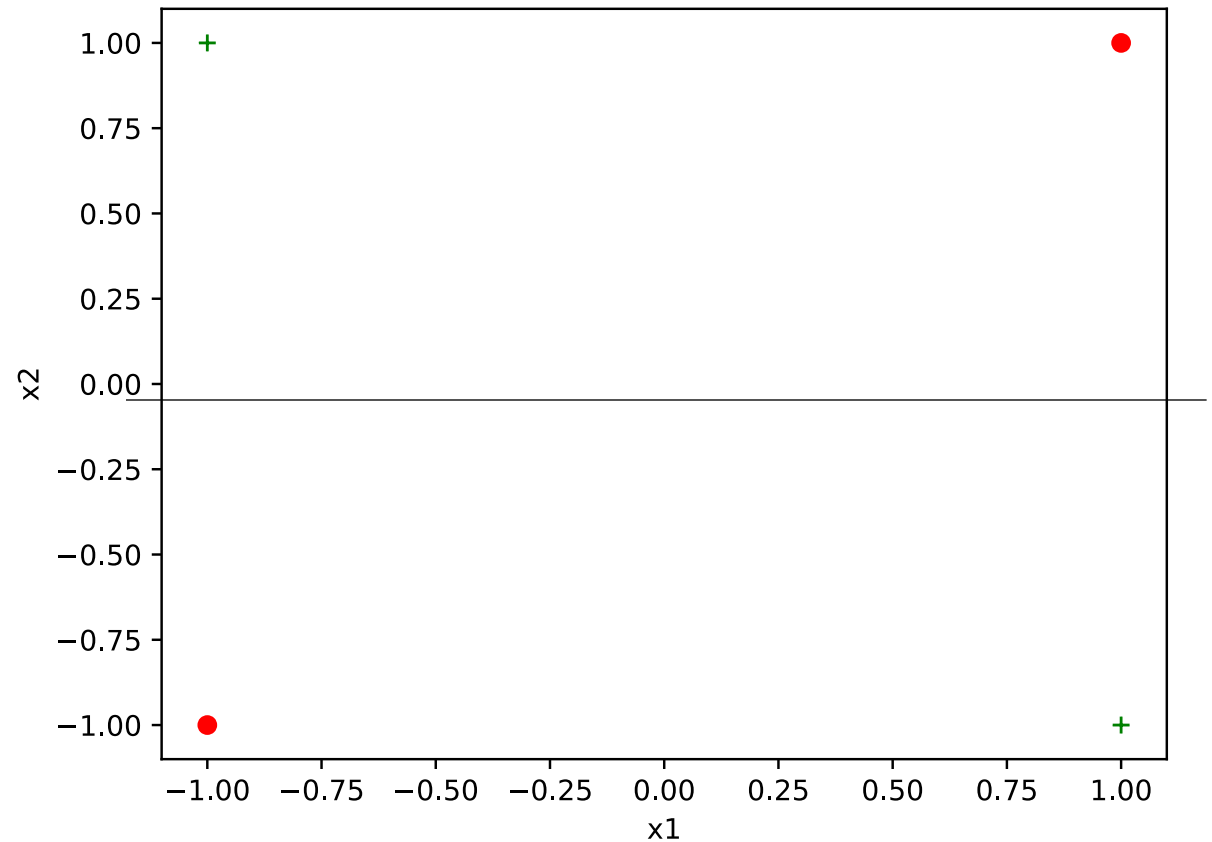
The XOR problem:



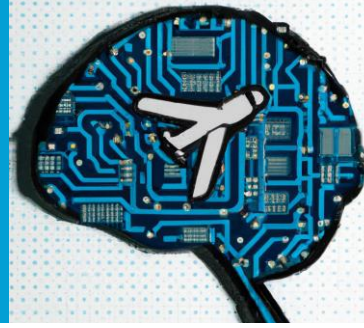
Limits of the perceptron



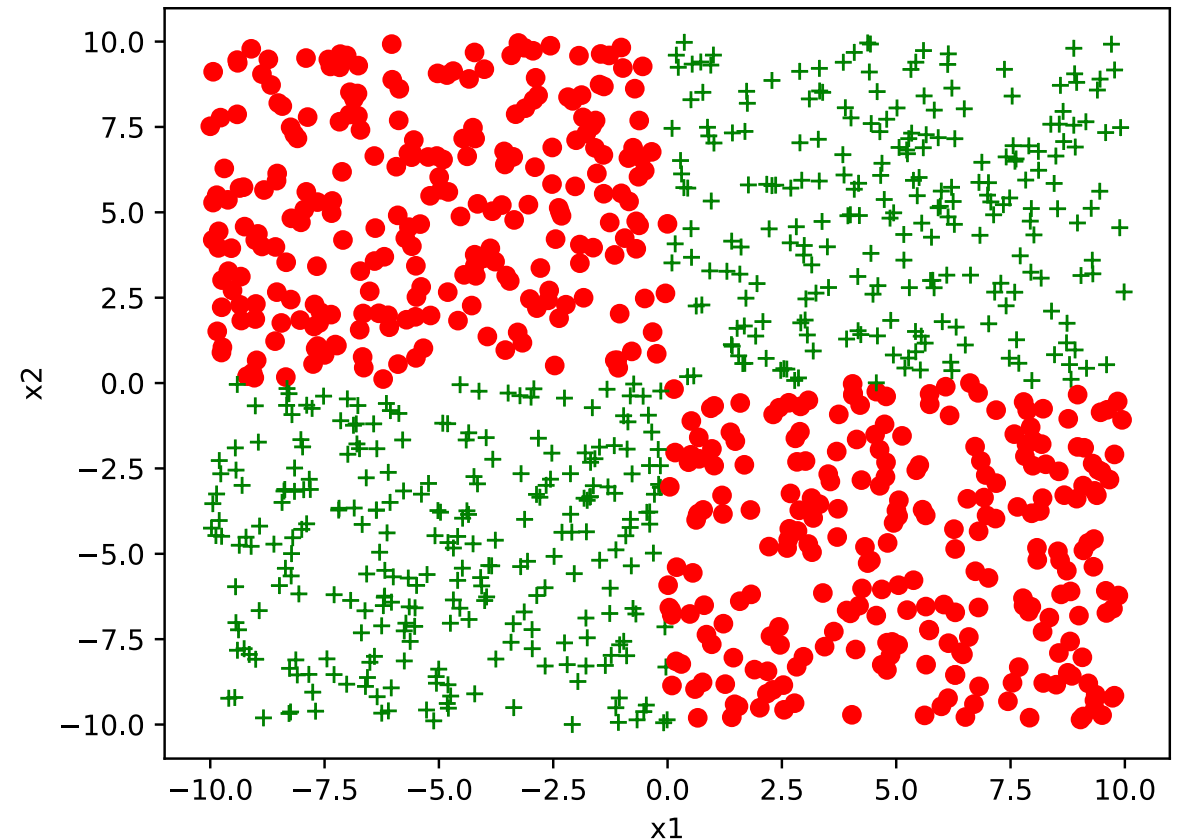
The XOR problem:



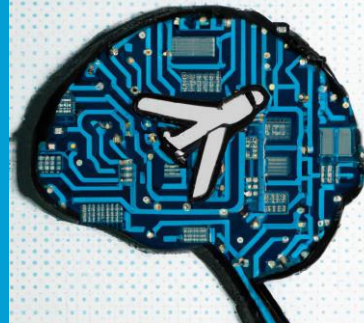
Limits of the perceptron



The XOR quadrant problem:

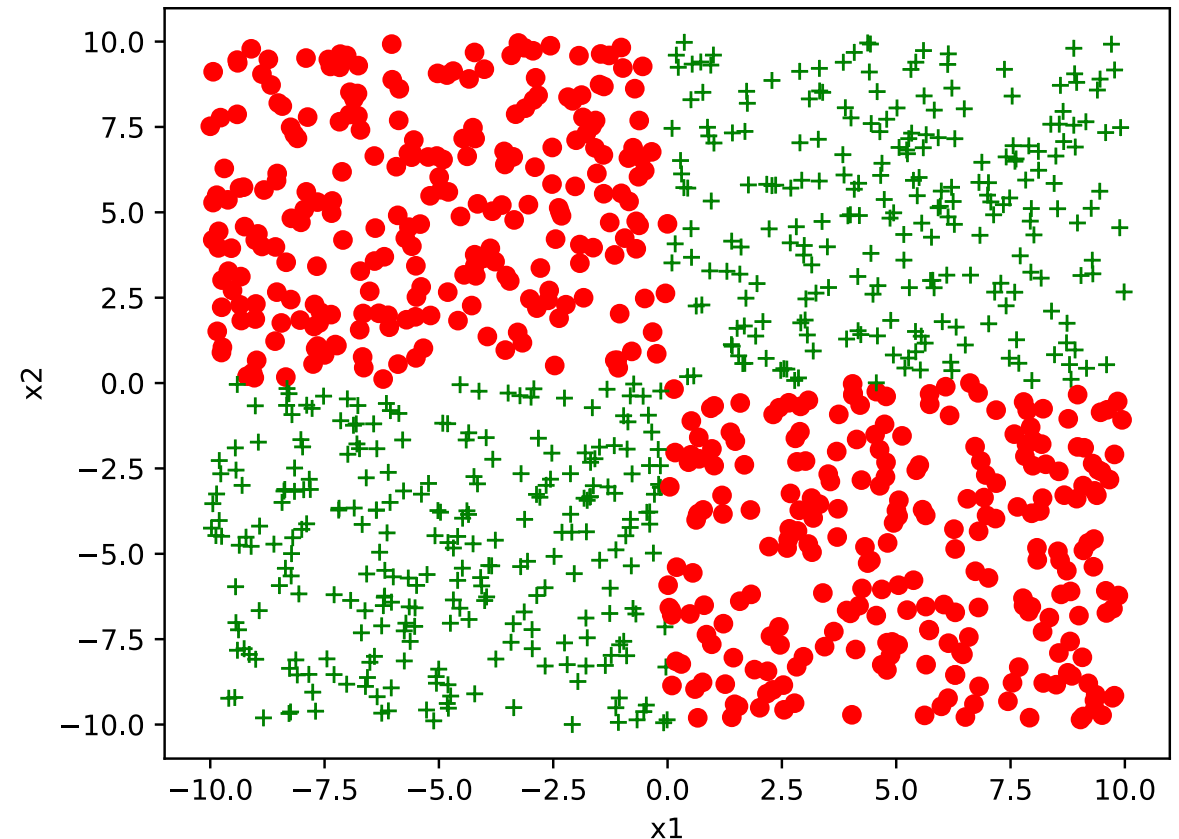


Limits of the perceptron

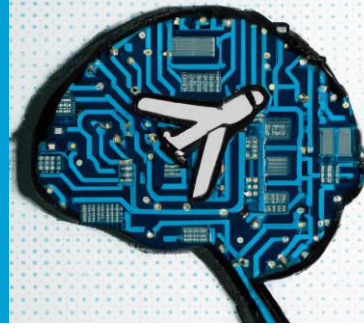


The XOR quadrant problem:

What if we redefine the inputs
as: $x'_1 = x_1 x_2$?



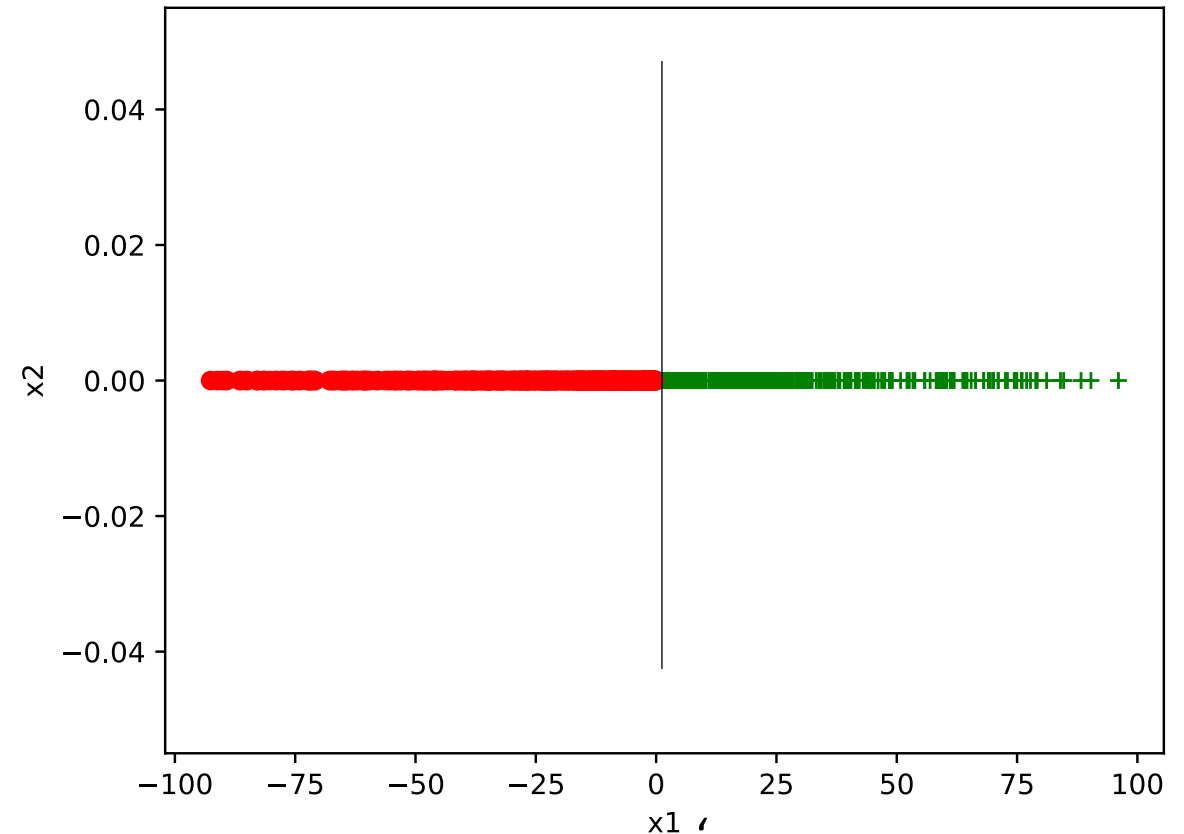
Limits of the perceptron

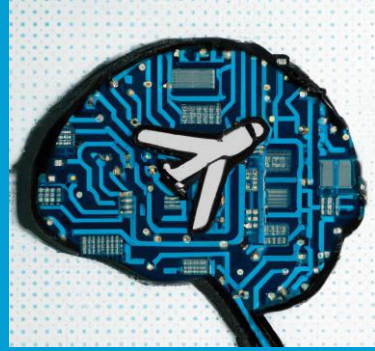


The XOR quadrant problem:

What if we redefine the inputs
as: $x'_1 = x_1 x_2$?

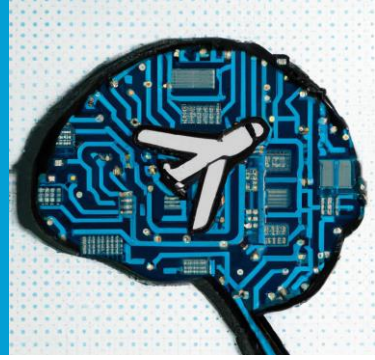
Representation matters!



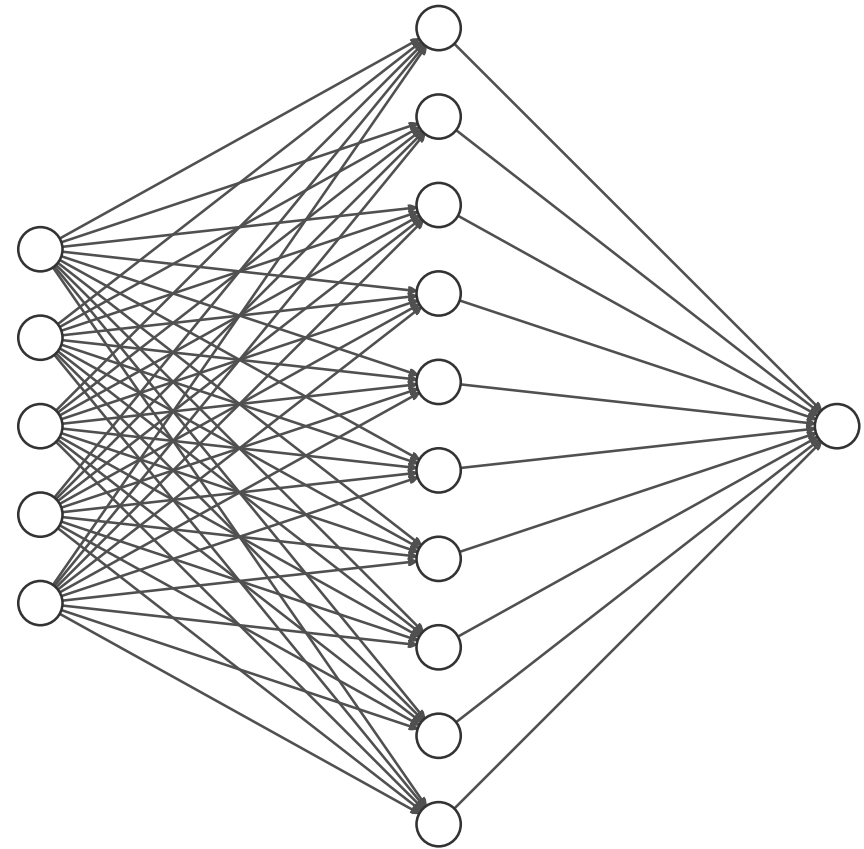


Multi-layer perceptrons

Multi-layer perceptrons



Can represent nonlinear functions.

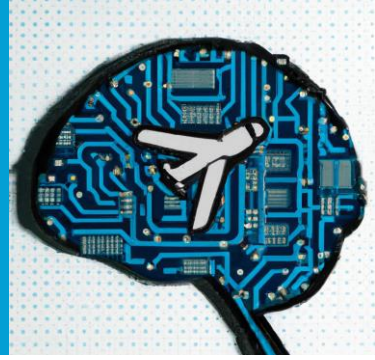


Input Layer $\in \mathbb{R}^5$

Hidden Layer $\in \mathbb{R}^{10}$

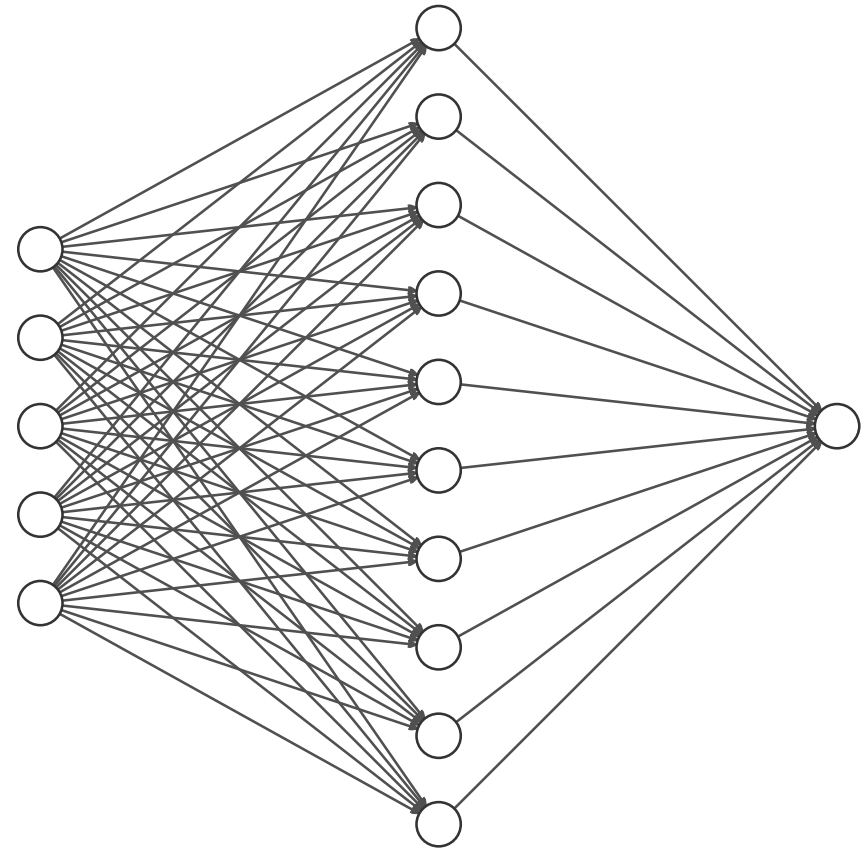
Output Layer $\in \mathbb{R}^1$

Multi-layer perceptrons



Can represent nonlinear functions.

What if all neurons use the linear activation function $f(a) = a$?

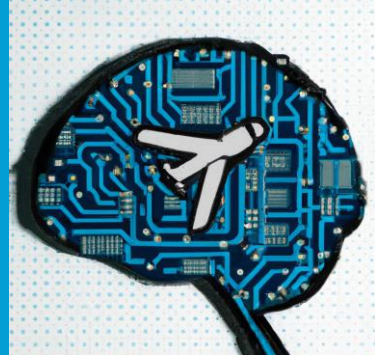


Input Layer $\in \mathbb{R}^5$

Hidden Layer $\in \mathbb{R}^{10}$

Output Layer $\in \mathbb{R}^1$

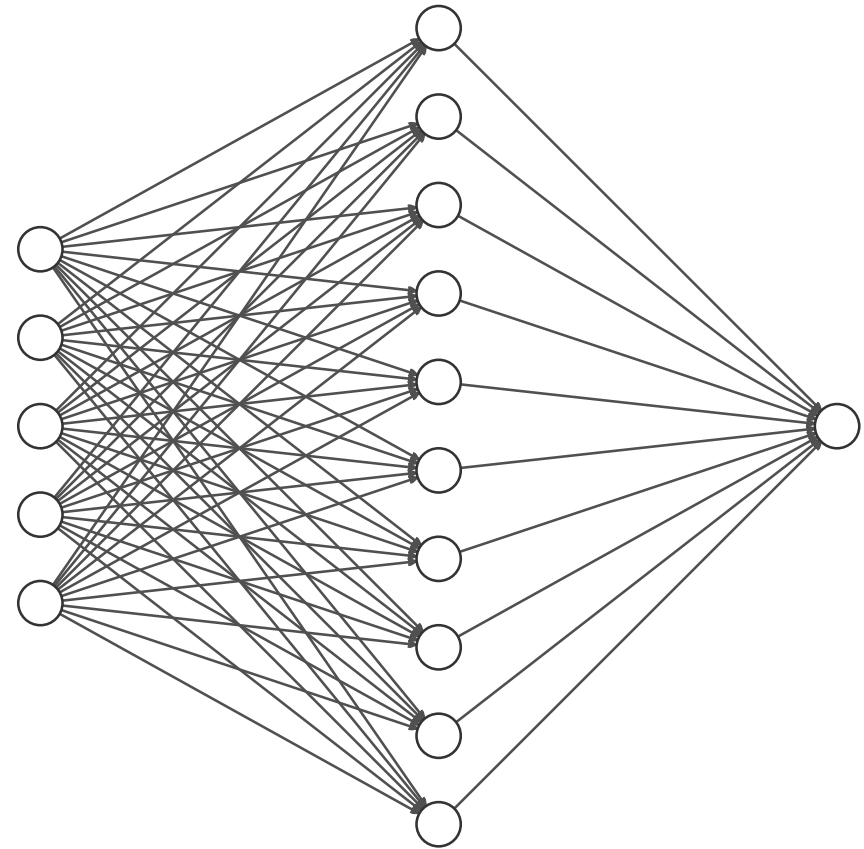
Multi-layer perceptrons



Can represent nonlinear functions.

What if all neurons use the linear activation function $f(a) = a$?

$$\begin{aligned}\vec{h} &= W_{ih}\vec{i} \\ \vec{o} &= W_{ho}\vec{h} \\ \vec{o} &= W_{ho}W_{ih}\vec{i} = W_{io}\vec{i}\end{aligned}$$

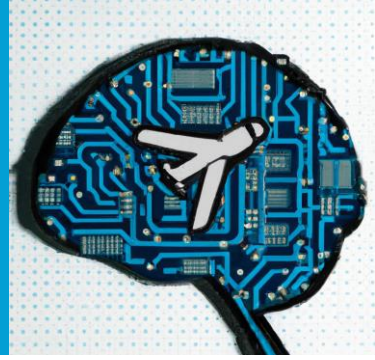


Input Layer $\in \mathbb{R}^5$

Hidden Layer $\in \mathbb{R}^{10}$

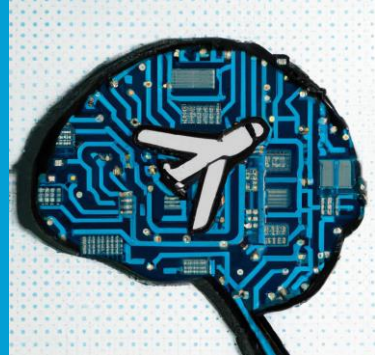
Output Layer $\in \mathbb{R}^1$

An MLP can model any function



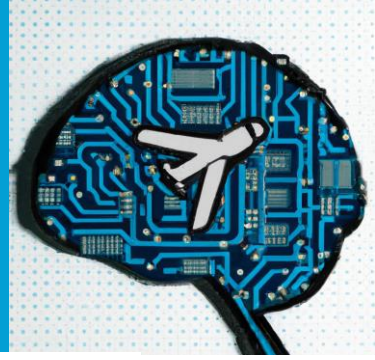
Multi-layer perceptrons can model any function (Cybenko, 1989, Kurt, 1991)

An MLP can model any function



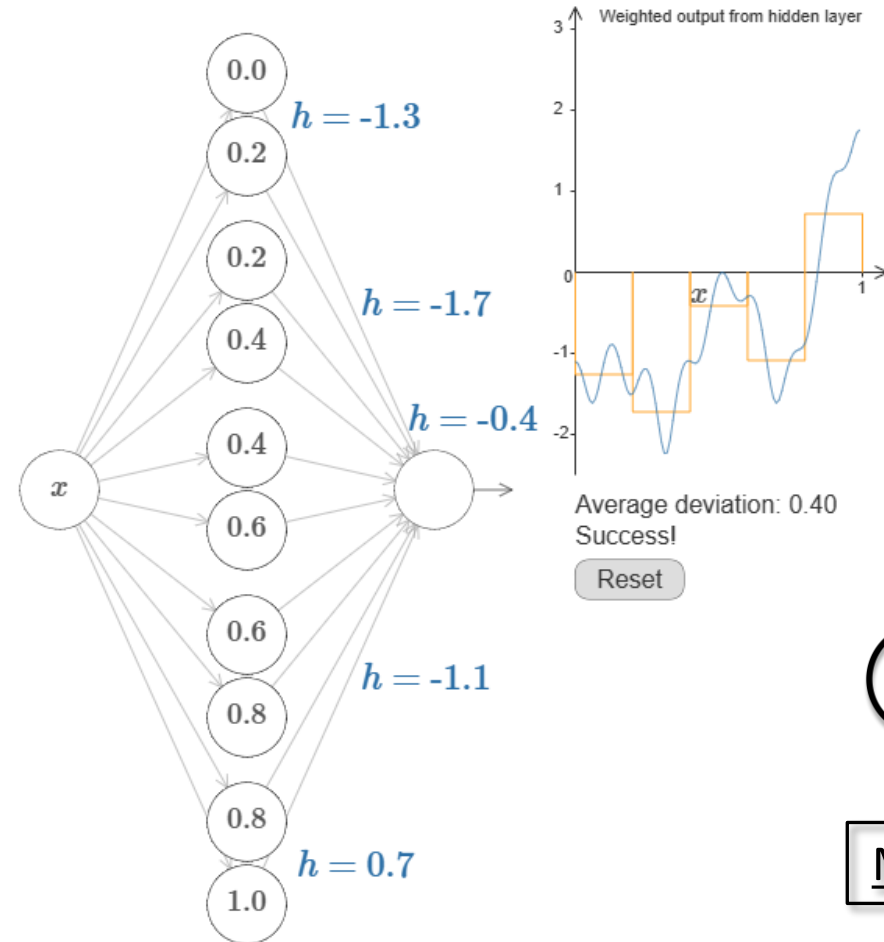
Multi-layer perceptrons can model any function (Cybenko, 1989, Kurt, 1991) – given infinitely many hidden neurons.

An MLP can model any function



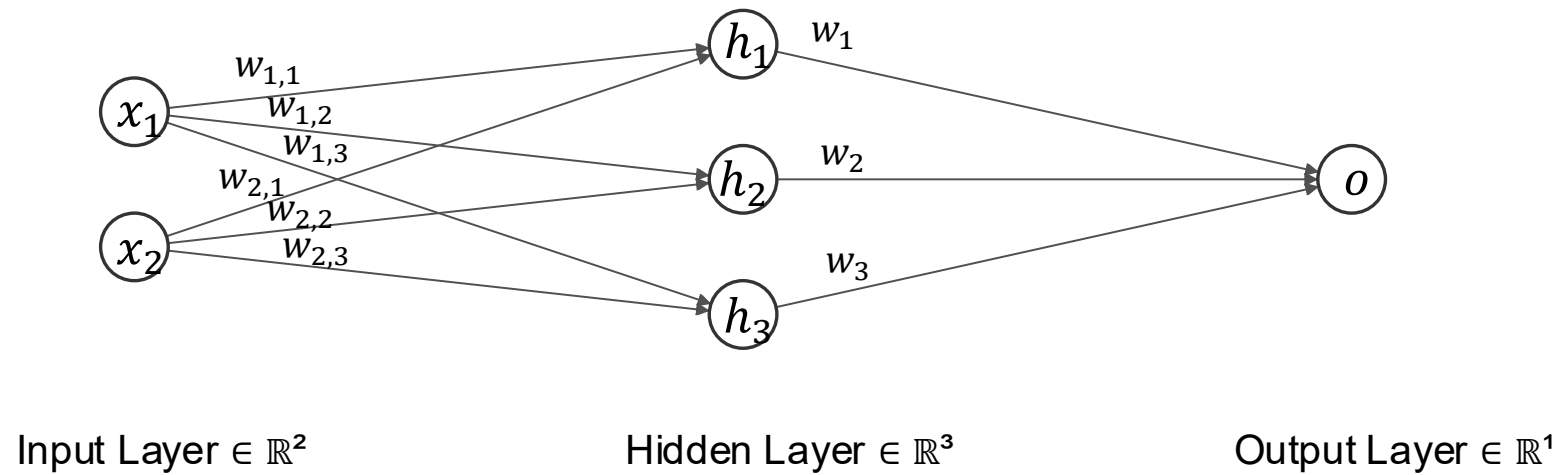
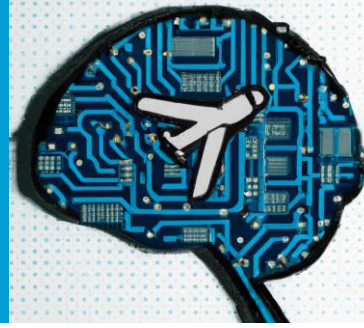
Multi-layer perceptrons can model any function (Cybenko, 1989, Kurt, 1991) – given infinitely many hidden neurons.

The proof relies on individual neurons covering small parts of the possibly highly nonlinear function.

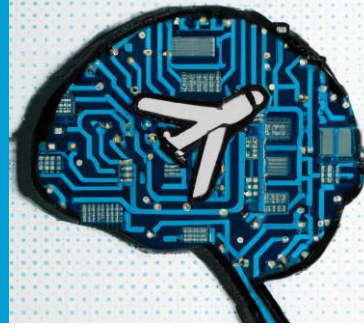


[More info](#)

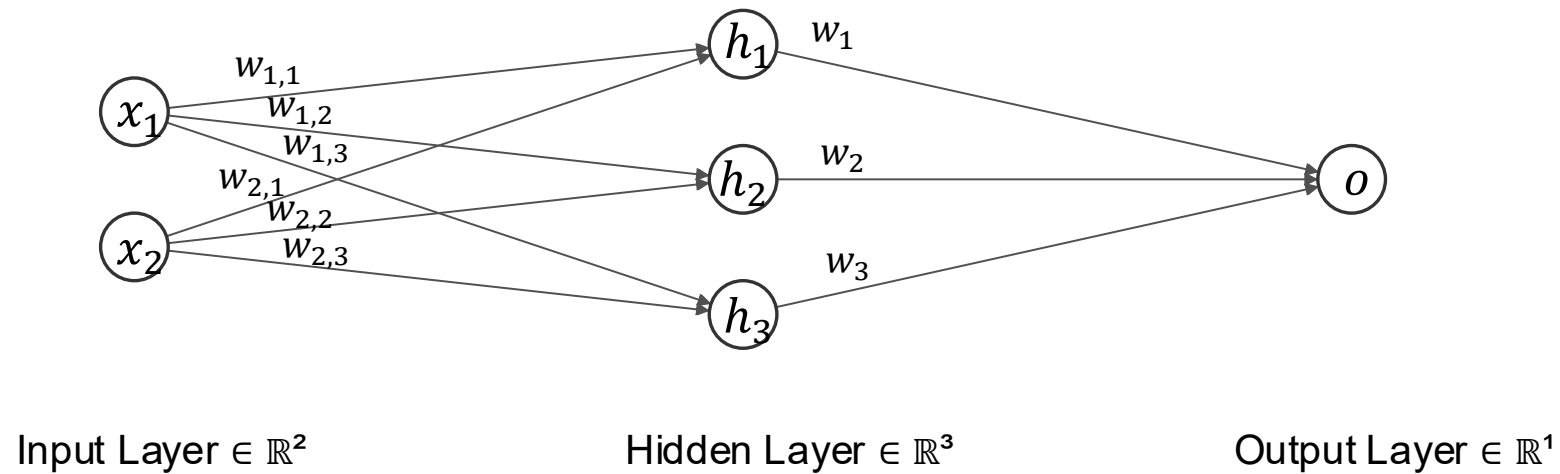
How to train an MLP?



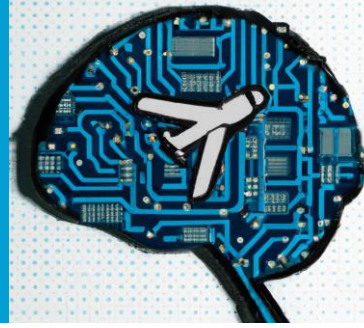
How to train an MLP?



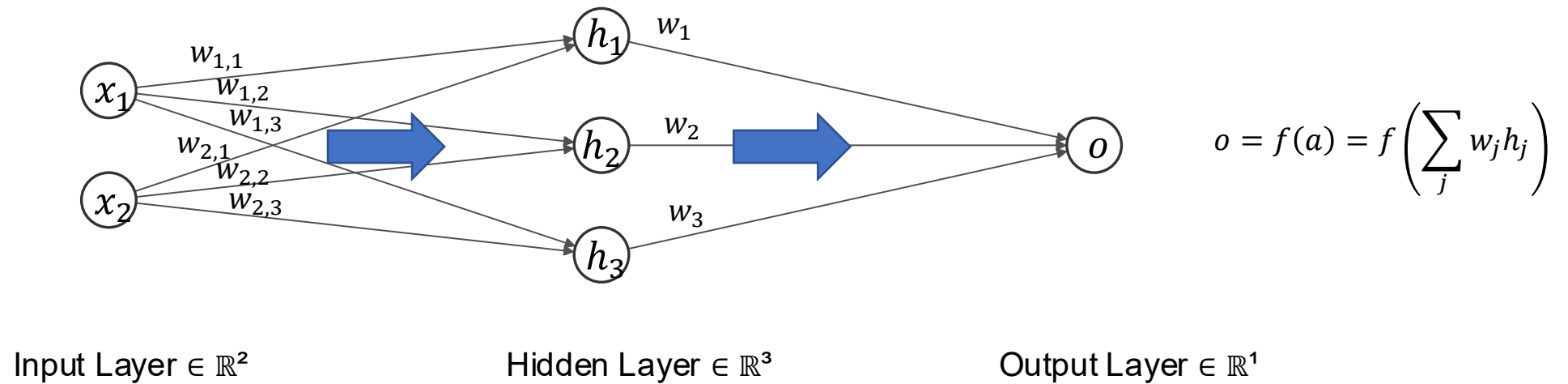
Backpropagation



How to train an MLP?



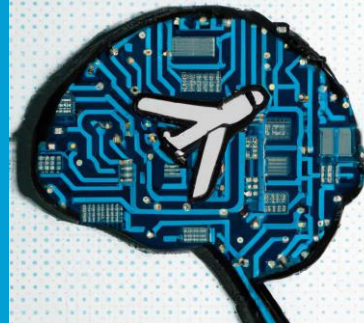
Backpropagation: Forward pass



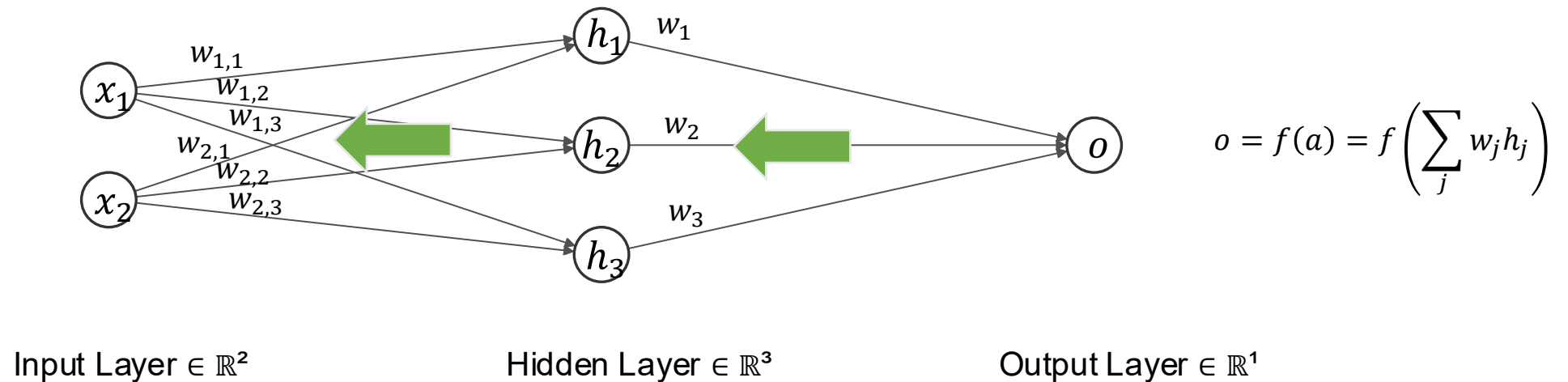
$$o = f(a) = f\left(\sum_j w_j h_j\right)$$

$$h_i = f(a_i) = f\left(\sum_j w_{j,i} x_j\right)$$

How to train an MLP?



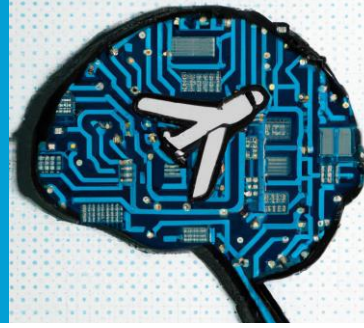
Backpropagation: Backward pass



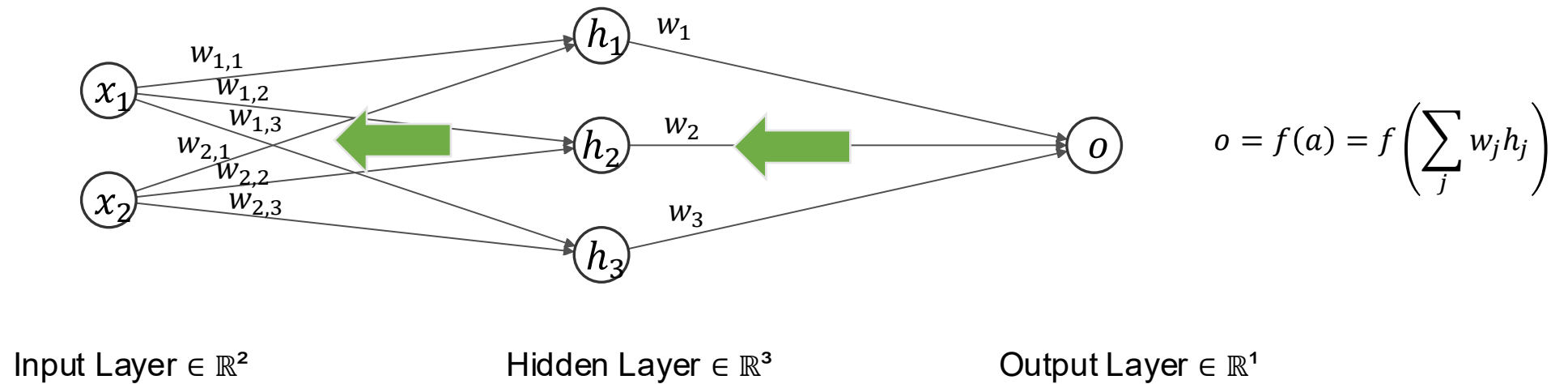
$$o = f(a) = f\left(\sum_j w_j h_j\right)$$

$$h_i = f(a_i) = f\left(\sum_j w_{j,i} x_j\right)$$

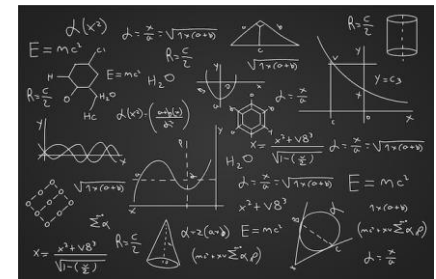
How to train an MLP?



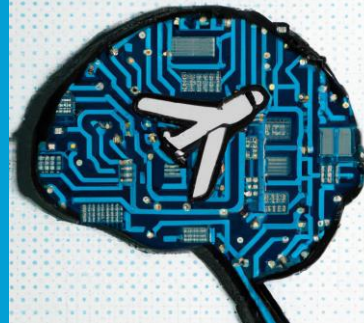
Backpropagation: Backward pass



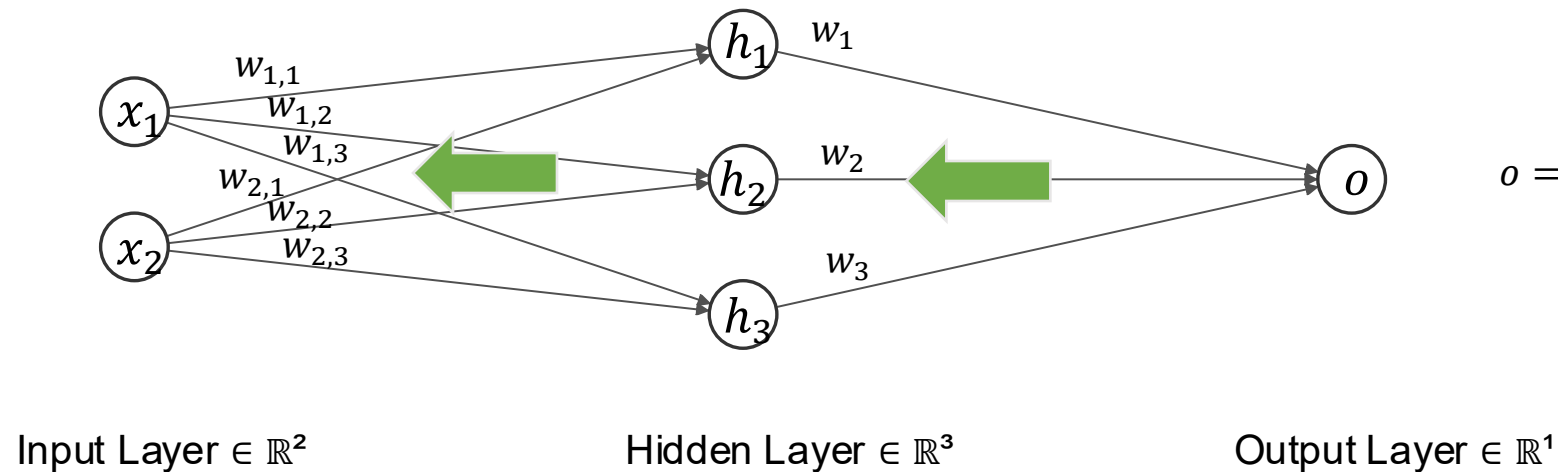
$$h_i = f(a_i) = f\left(\sum_j w_{j,i} x_j\right)$$



How to train an MLP?



Backpropagation: Backward pass



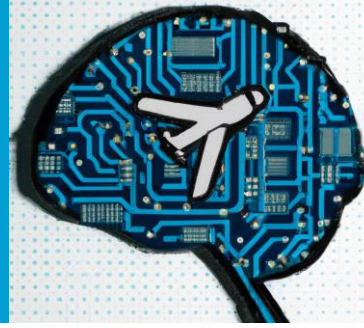
$$e = (o - t)^2$$

$$o = f(a) = f\left(\sum_j w_j h_j\right)$$

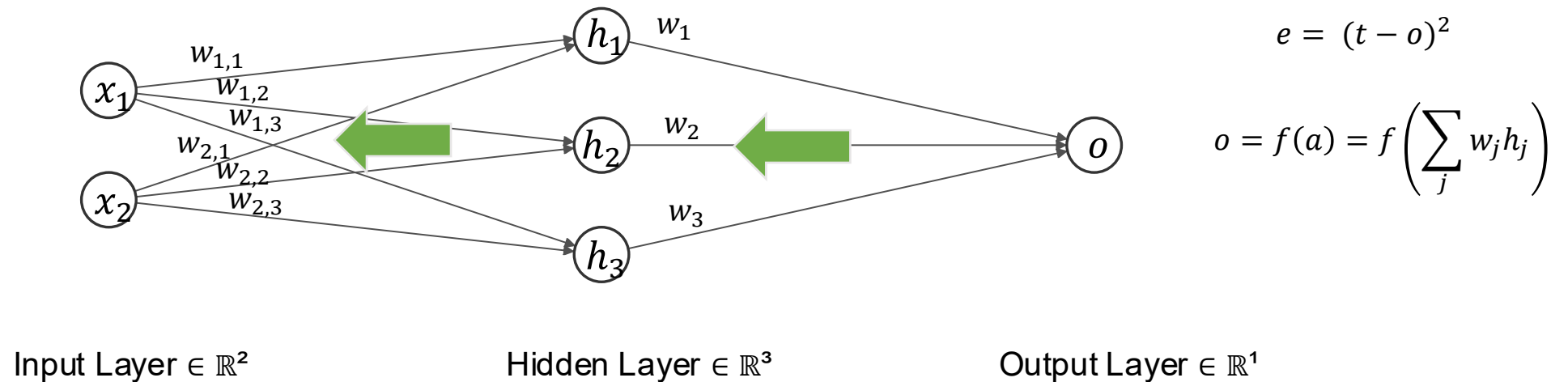
An example gradient using the chain rule:

$$\frac{\partial e}{\partial w_{2,3}} = \frac{\partial e}{\partial o} \frac{\partial o}{\partial a} \frac{\partial a}{\partial h_3} \frac{\partial h_3}{\partial a_3} \frac{\partial a_3}{\partial w_{2,3}} = 2(o - t)f'(a)w_3f'(a_3)x_2$$

How to train an MLP?



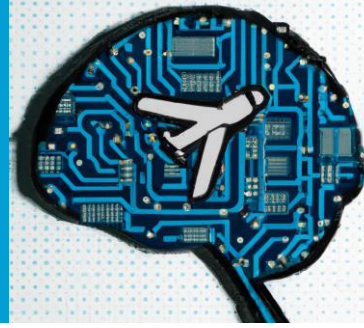
Backpropagation: Backward pass



An example gradient using the chain rule: To calculate it, we need values from the [forward pass](#).

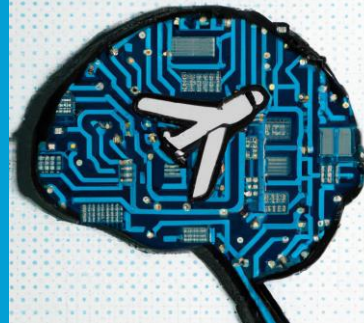
$$\frac{\partial e}{\partial w_{2,3}} = \frac{\partial e}{\partial o} \frac{\partial o}{\partial a} \frac{\partial a}{\partial h_3} \frac{\partial h_3}{\partial a_3} \frac{\partial a_3}{\partial w_{2,3}} = 2(o - t)f'(a)w_3f'(a_3)x_2$$

Stochastic gradient descent (SGD)



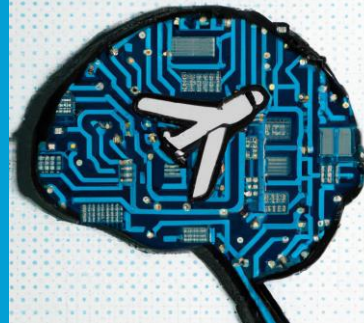
- Instead of feeding the network with the entire training data set of size N for each epoch, a smaller batch of size B is used.

Stochastic gradient descent (SGD)



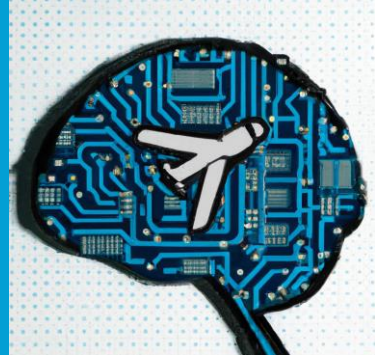
- Instead of feeding the network with the entire training data set of size N for each epoch, a smaller batch of size B is used.
- One epoch then requires $I = \frac{N}{B}$ iterations.

Stochastic gradient descent (SGD)



- Instead of feeding the network with the entire training data set of size N for each epoch, a smaller batch of size B is used.
- One epoch then requires $I = \frac{N}{B}$ iterations.
- Typically, gradients are averaged over the batch for smoother optimization.

Backpropagation implementation



Require: batch_size, n_epochs, n_iterations, learning_rate, n_hidden

Initialize the MLP (n_hidden)

for epoch 1:n_epochs

 for iteration 1:n_iterations

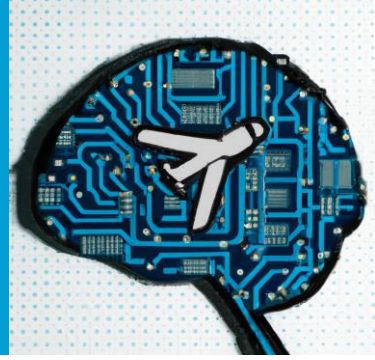
 Get a batch of size B

 for sample 1:B

 Perform forward pass and backward pass, accumulating the gradients for each model parameter (weight)

 Update the parameters with the average gradient (learning_rate)

Backpropagation implementation



Require: batch_size, n_epochs, n_iterations, learning_rate, n_hidden

Initialize the MLP (n_hidden)

for epoch 1:n_epochs

 for iteration 1:n_iterations

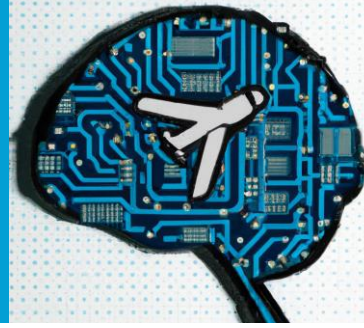
 Get a batch of size B

 for sample 1:B

 Perform forward pass and backward pass, accumulating the gradients for each model parameter (weight)

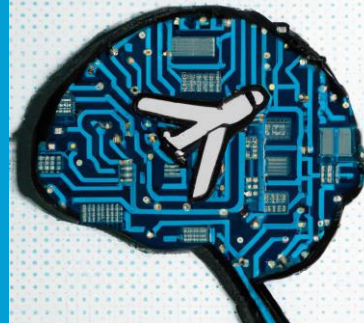
 Update the parameters with the average gradient (learning_rate)

Weight initialization



- Weights should not be too large, nor too small 😊
- Dangers:
 - Blow-up or dwindling of activation during forward pass.
 - Vanishing gradients during backward pass.
 - If weights are all equal, they will remain equal with deterministic weight changes in training.

Weight initialization



- Weights should not be too large, nor too small 😊

- Dangers:

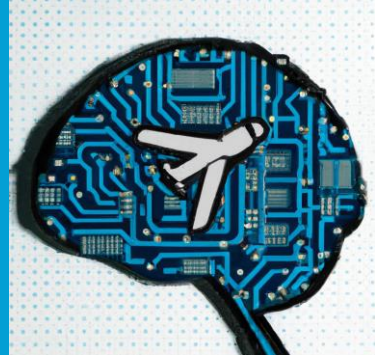
Note:

Always study the statistics of your inputs (are they large or small, do they vary a lot in magnitude?)

Typically, it is good to normalize the inputs (zero mean, unit standard deviation).

Also study the outputs: are they supposed to be very large? Can they be remapped to a different scale, closer to the scale of the inputs?

Weight initialization

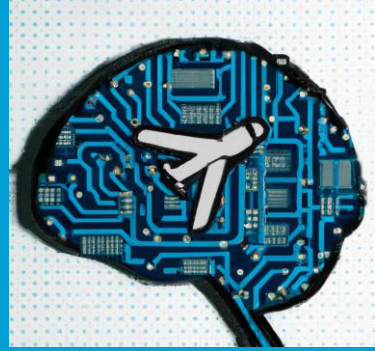


- Employ randomness
- Take into account the “fan in” of a neuron

For example:

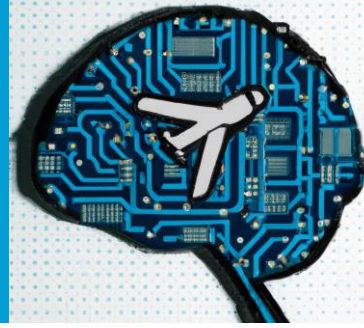
$$w \sim U\left(-\frac{1}{M}, \frac{1}{M}\right)$$

Where M is the number of incoming neurons.



Machine learning fundamentals (II)

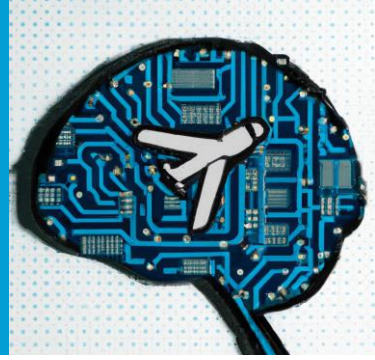
Capacity



The “capacity” of a model captures how wide a variety of functions it can model.

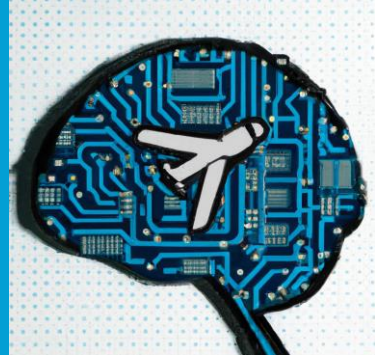
- Perceptrons can model only linear functions
- Multi-layer perceptrons can learn nonlinear functions
- Networks with more layers / neurons have a larger capacity

Capacity



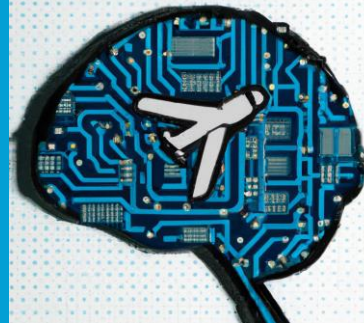
Should we always pick the biggest possible neural network?

Capacity



A **low capacity** may lead to **underfitting** – poor performance on the training set. The model has a high **bias** (may fail to capture the relationship between inputs and outputs).

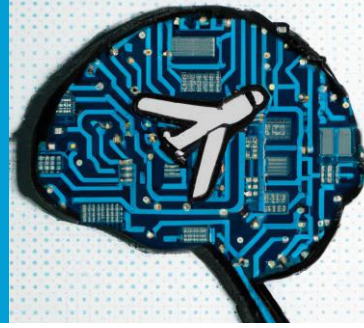
Capacity



A **low capacity** may lead to **underfitting** – poor performance on the training set. The model has a high **bias** (may fail to capture the relationship between inputs and outputs).

A **high capacity** may lead to **overfitting** – a great performance on the training set, but bad performance on the test set. The model has a high **variance** (different samplings of the training inputs may lead to very different parameters).

Capacity

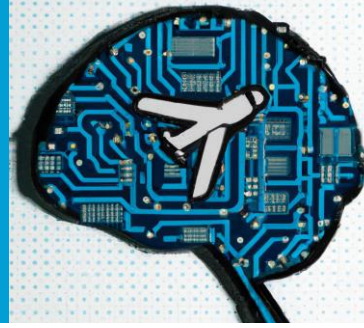


A **low capacity** may lead to **underfitting** – poor performance on the training set. The model has a high **bias** (may fail to capture the relationship between inputs and outputs).

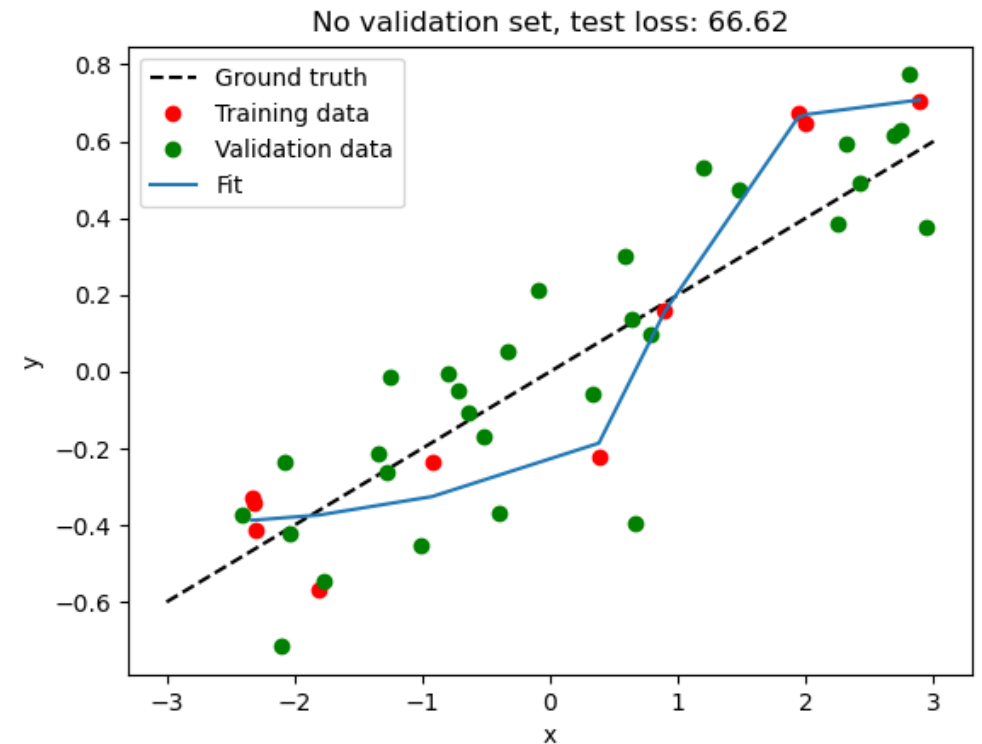
A **high capacity** may lead to **overfitting** – a great performance on the training set, but bad performance on the test set. The model has a high **variance** (different samplings of the training inputs may lead to very different parameters).

Selecting the right model means exploring the **bias-variance trade-off**.

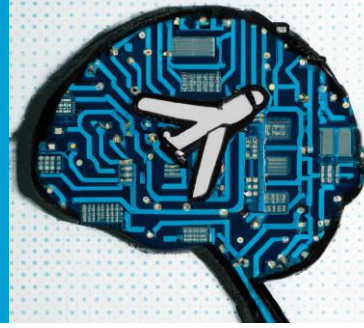
Early stopping with a validation set



Neural networks can model complex functions. If training goes on for too long, the network may start to *overfit* on the training data.



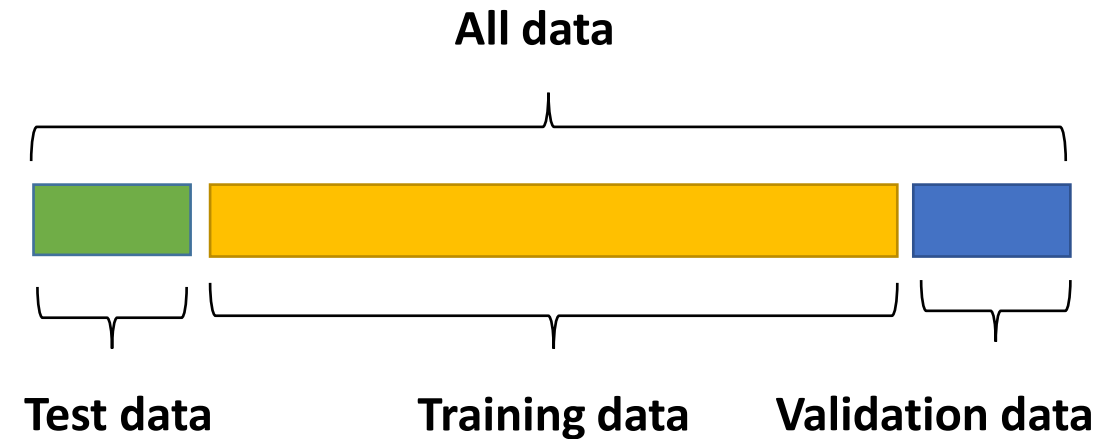
Early stopping with a validation set



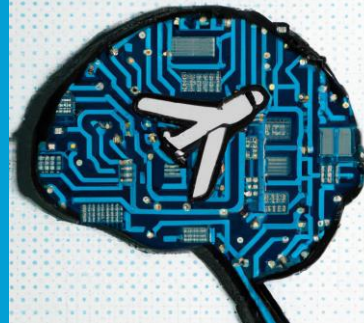
Neural networks can model complex functions. If training goes on for too long, the network may start to *overfit* on the training data.

A solution is to apply the network to a *validation set* and stop training when the validation error increases.

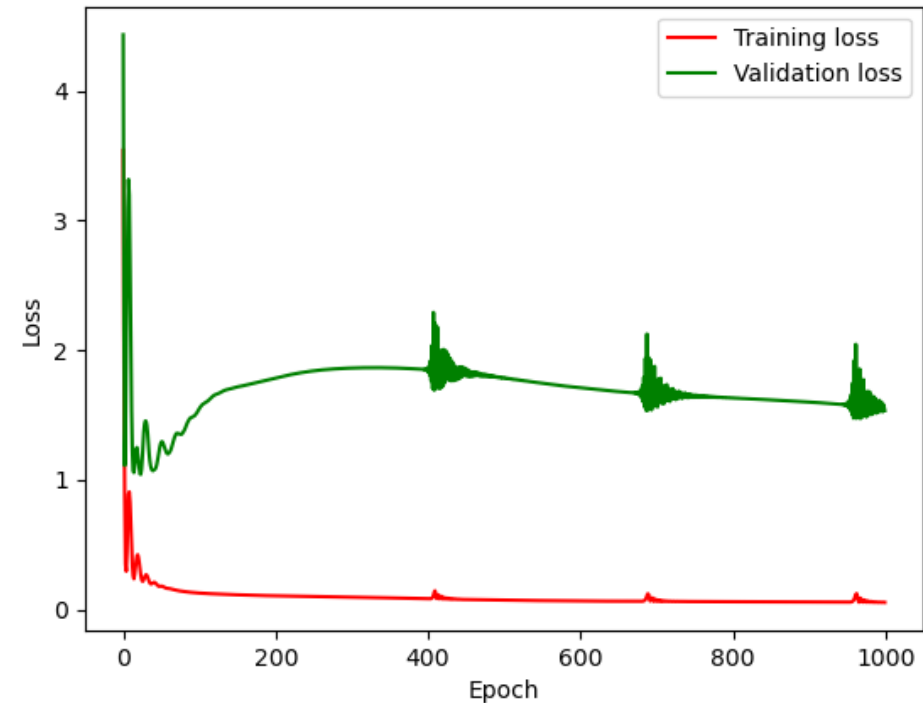
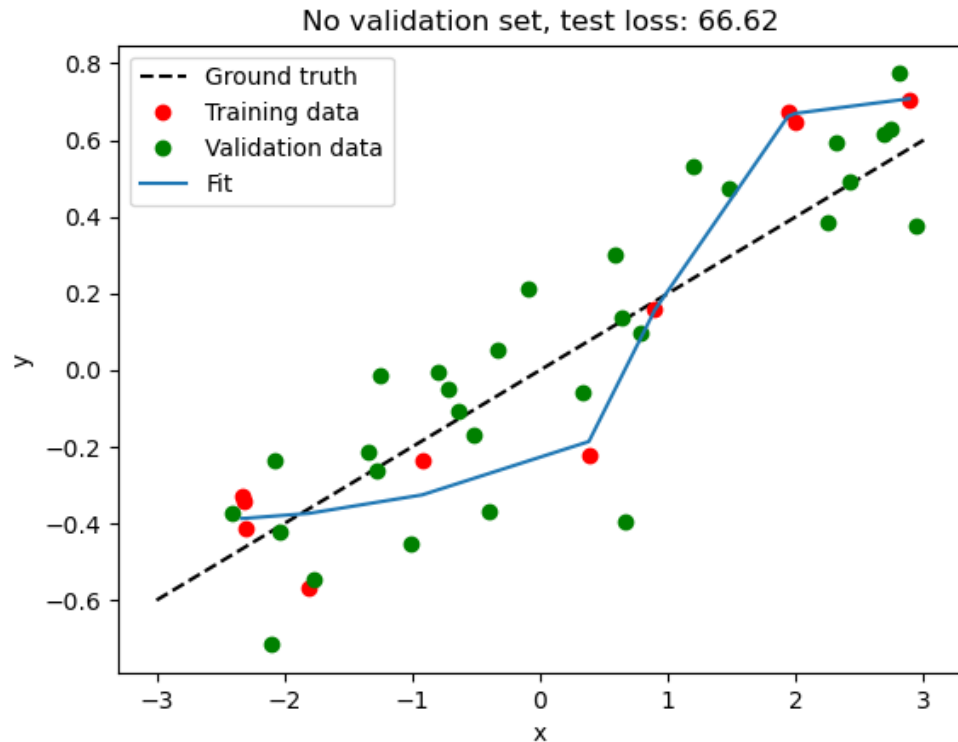
A typical data split:



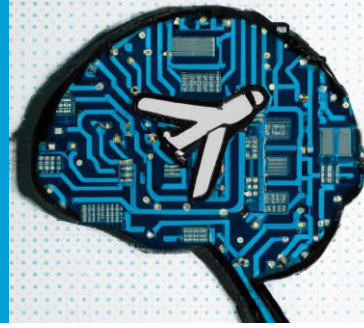
Early stopping with a validation set



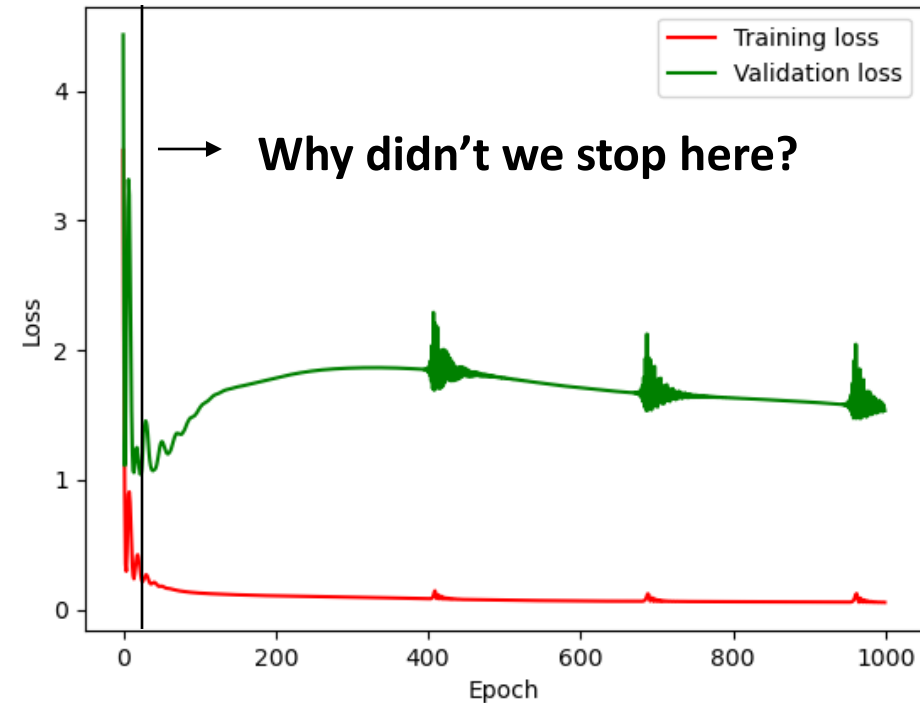
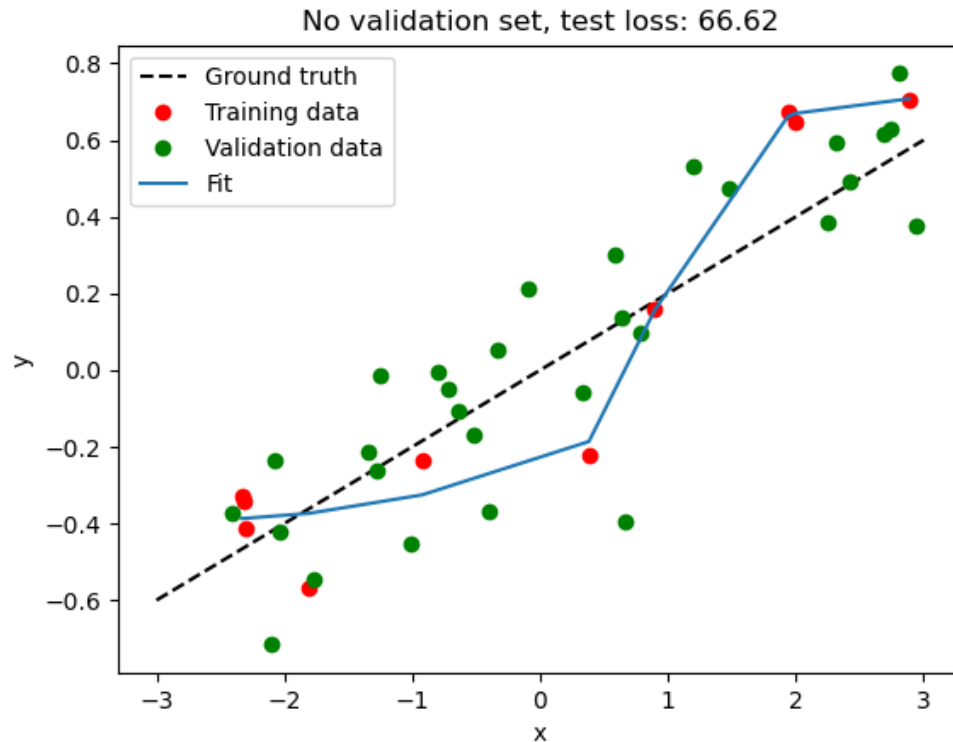
Training a linear function with a 100-hidden neuron MLP:



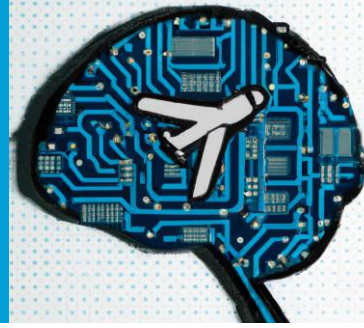
Early stopping with a validation set



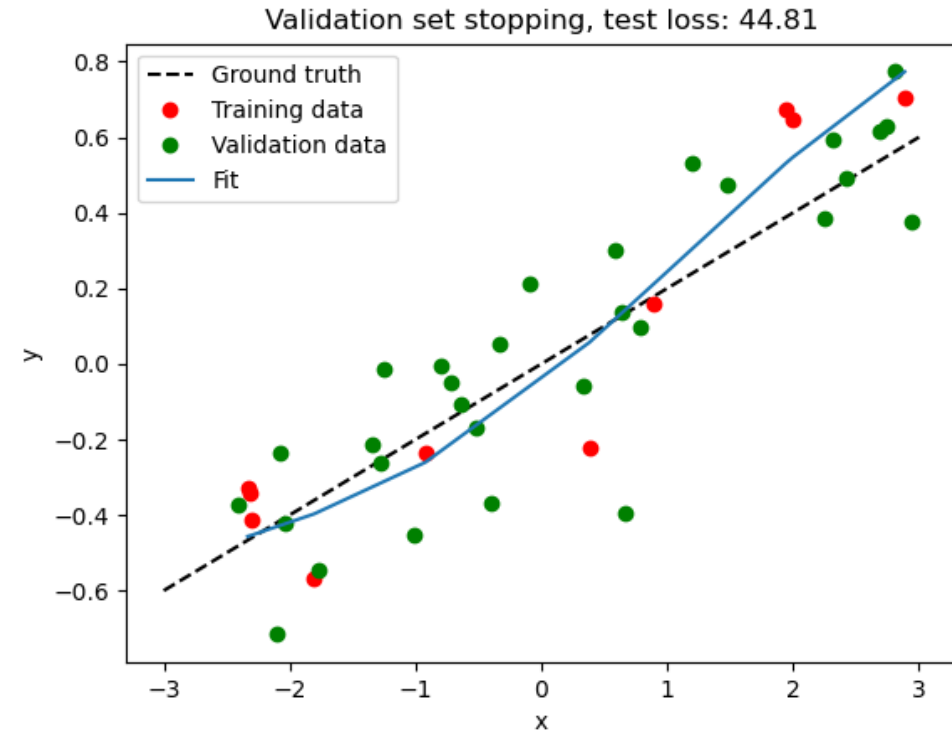
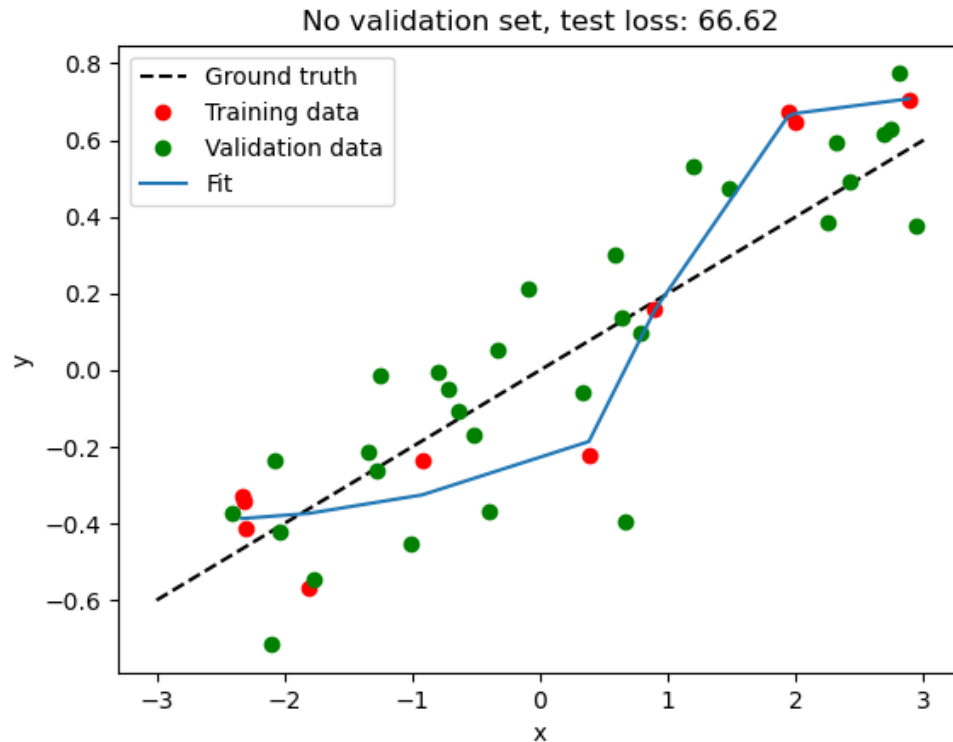
Training a linear function with a 100-hidden neuron MLP:

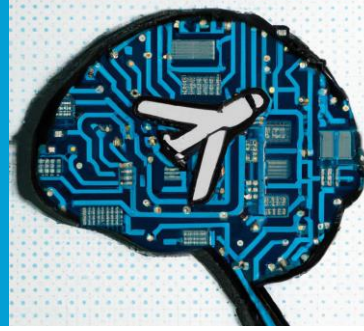


Early stopping with a validation set



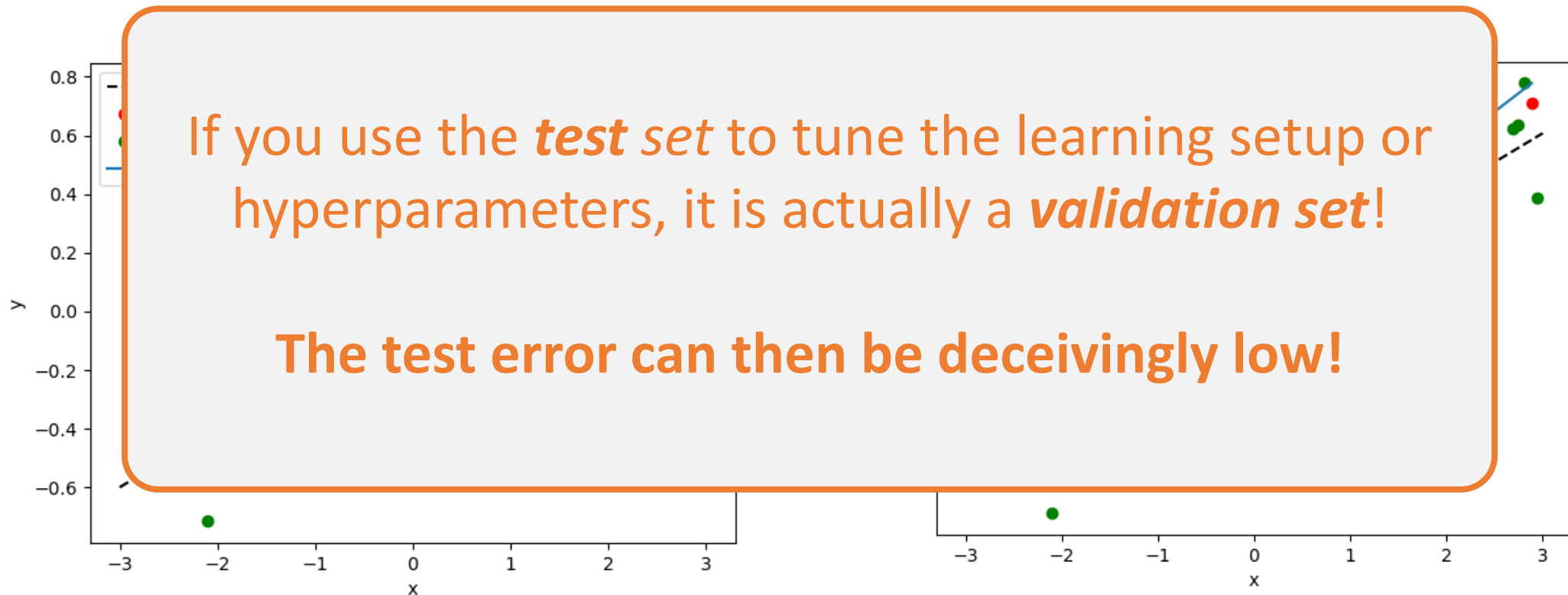
Training a linear function with a 100-hidden neuron MLP:



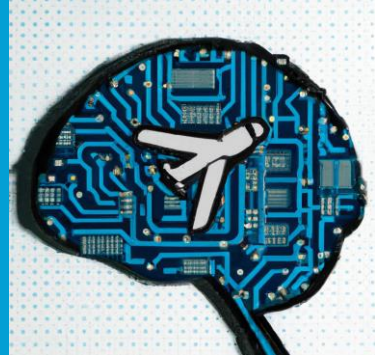


Early stopping with a validation set

Training a linear function with a 100-hidden neuron MLP:



K-fold cross validation

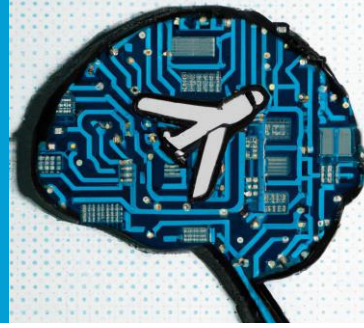


Often, benchmark datasets have a single training / test split of the data.

However, better insight into a learning algorithm's performance can be obtained by running multiple experiments, in which the training and test sets vary.

K-fold cross validation

K-fold cross validation

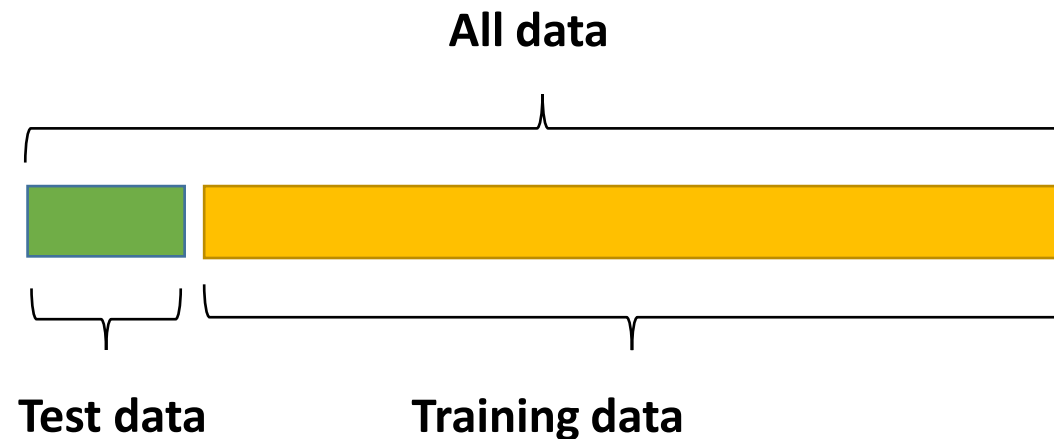


Often, benchmark datasets have a single training / test split of the data.

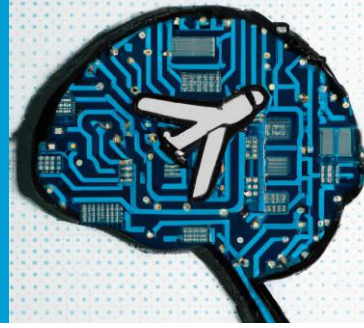
However, better insight into a learning algorithm's performance can be obtained by running multiple experiments, in which the training and test sets vary.

K-fold cross validation:

K=6, Fold 1



K-fold cross validation

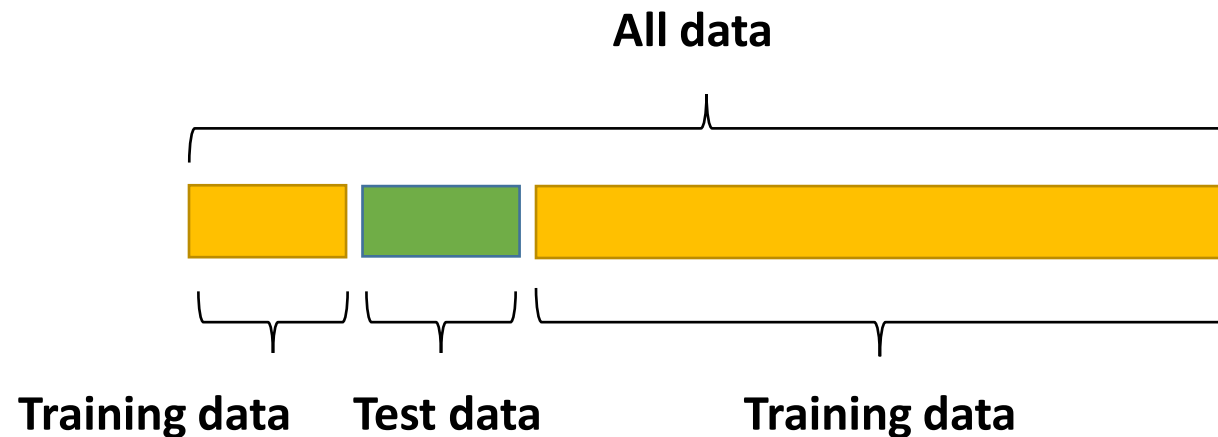


Often, benchmark datasets have a single training / test split of the data.

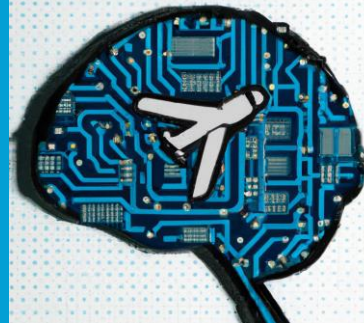
However, better insight into a learning algorithm's performance can be obtained by running multiple experiments, in which the training and test sets vary.

K-fold cross validation:

K=6, Fold 2



K-fold cross validation

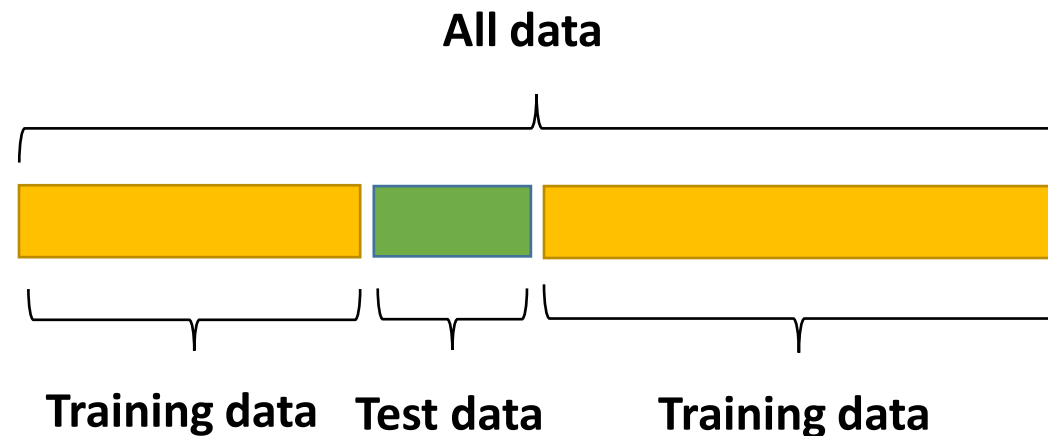


Often, benchmark datasets have a single training / test split of the data.

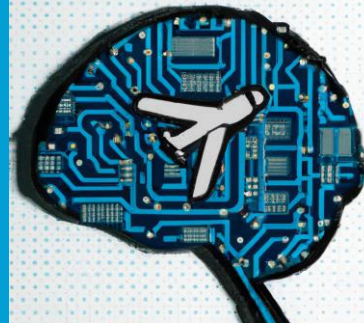
However, better insight into a learning algorithm's performance can be obtained by running multiple experiments, in which the training and test sets vary.

K-fold cross validation:

K=6, Fold 3



K-fold cross validation



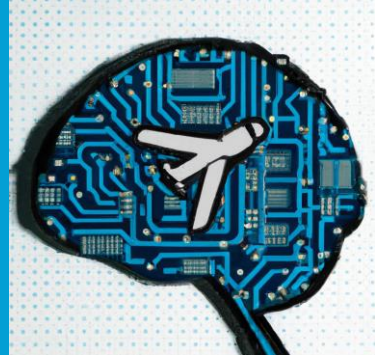
Often, benchmark datasets have a single training / test split of the data.

However, better insight into a learning algorithm's performance can be obtained by running multiple experiments, in which the training and test sets vary.

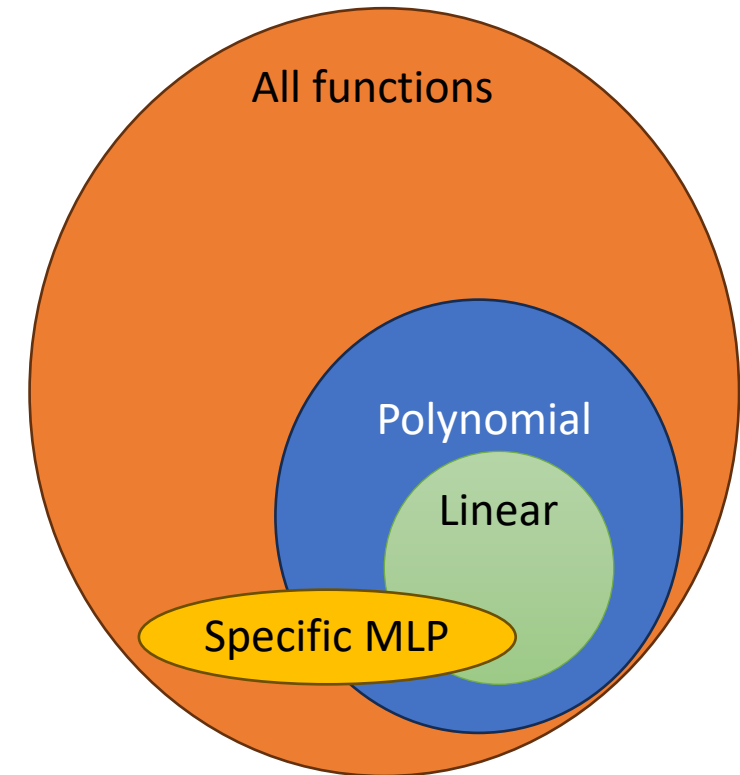
K-fold cross validation:

Etc.

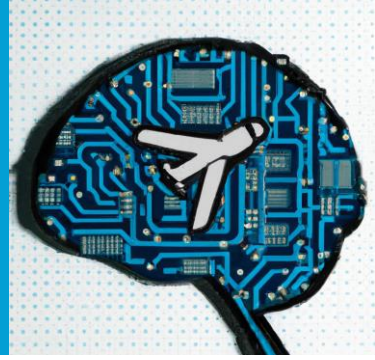
Hypothesis space



The capacity is determined by the “**hypothesis space**”, i.e., the set that the learning algorithm is allowed to select from.

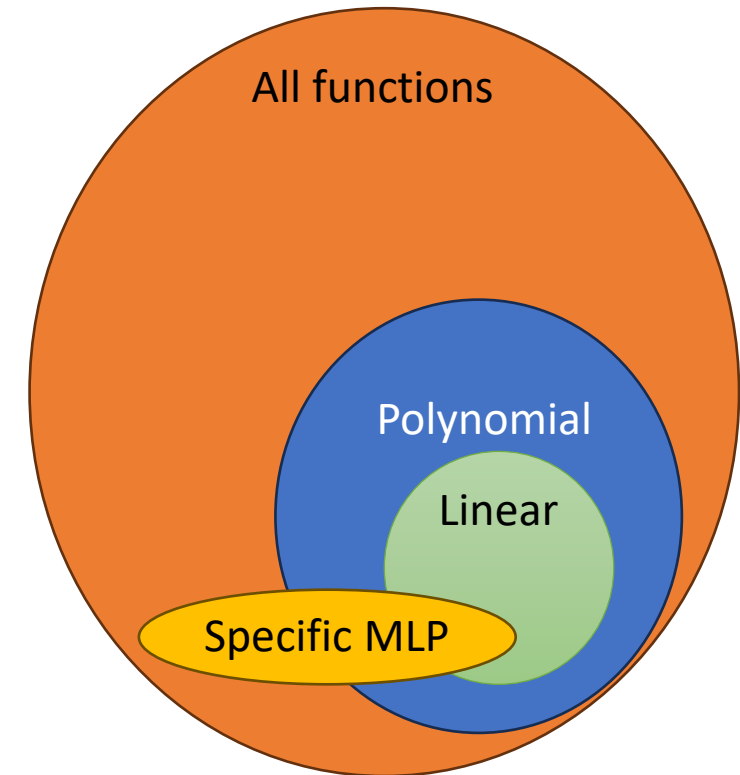


Hypothesis space

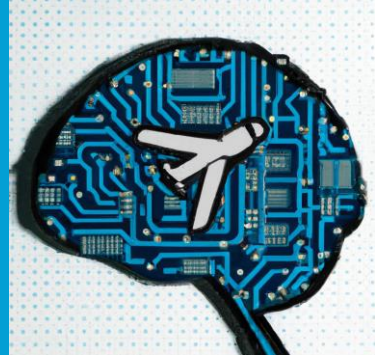


The capacity is determined by the “**hypothesis space**”, i.e., the set that the learning algorithm is allowed to select from.

What if there are multiple hypotheses that capture the data equally well?



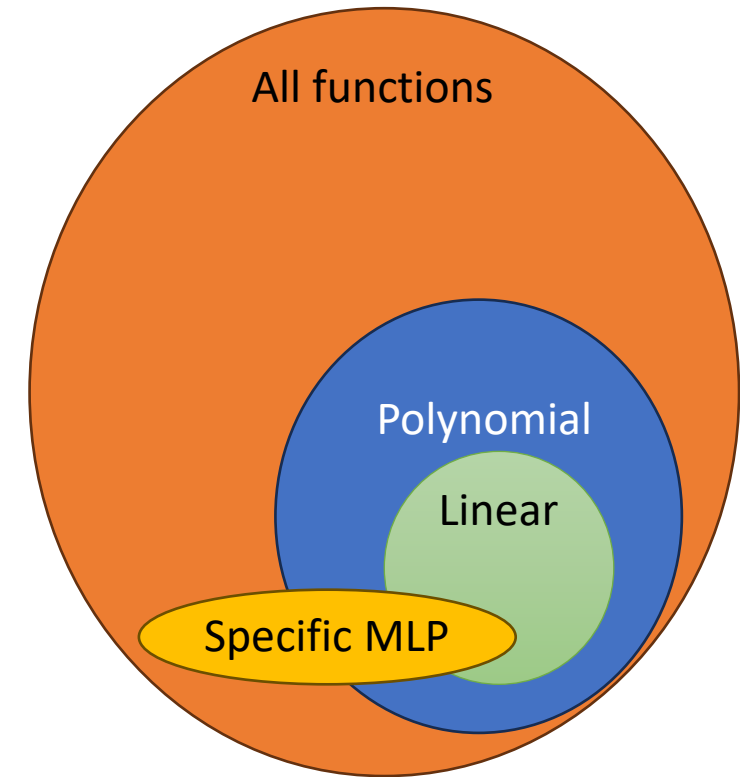
Hypothesis space



The capacity is determined by the “**hypothesis space**”, i.e., the set that the learning algorithm is allowed to select from.

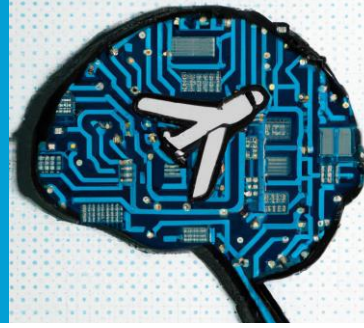
What if there are multiple hypotheses that capture the data equally well?

Occam's razor – parsimony principle



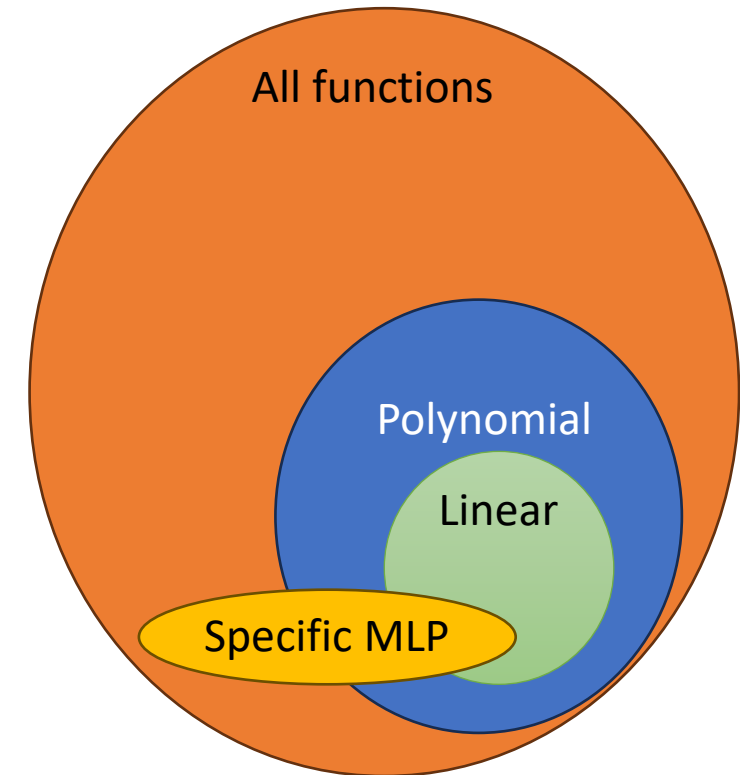
William of Ockham,
1287–1347, [Wikipedia](#)

Hypothesis space

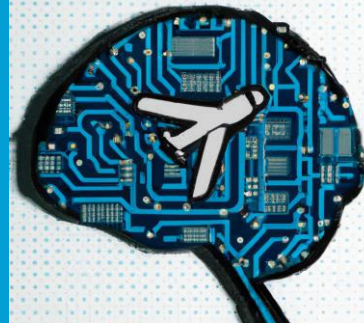


The hypothesis space is strongly related to the **inductive bias**

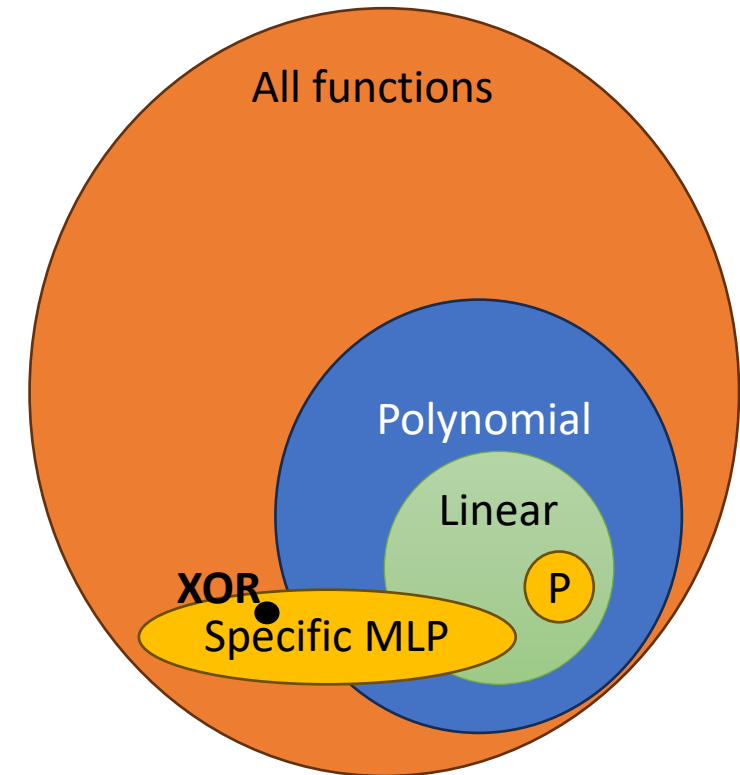
Inductive bias: The set of assumptions a learner makes for predicting the outputs of test samples.



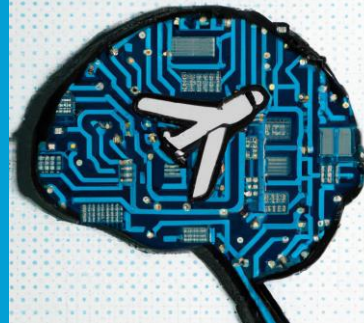
Hypothesis space



Learning may fail because the actual function is not in the chosen hypothesis space.

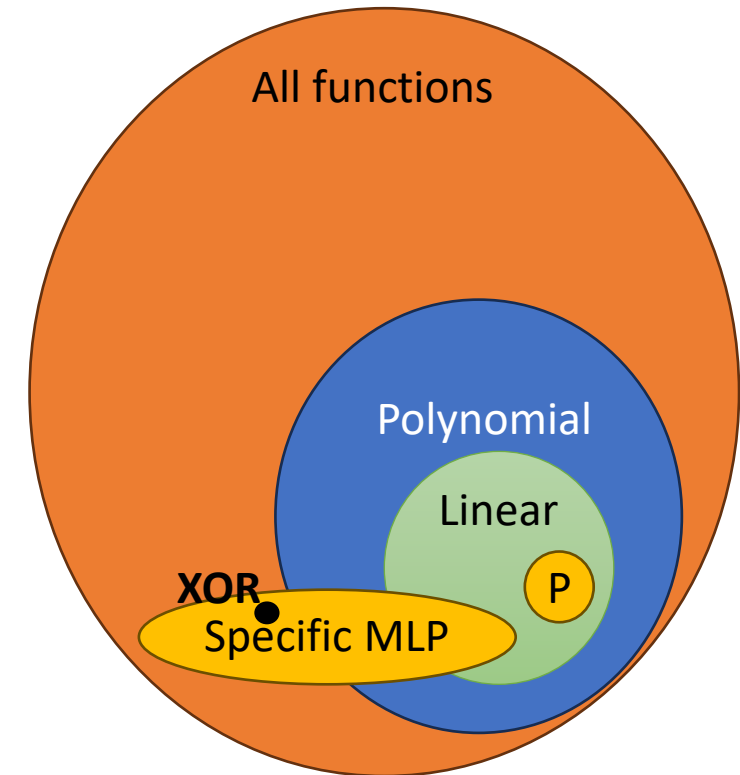


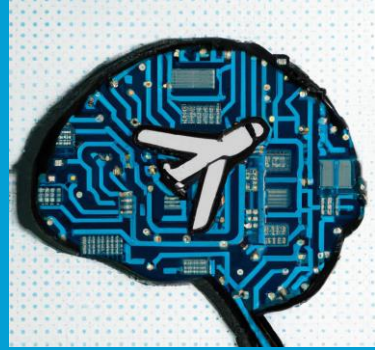
Hypothesis space



Learning may fail because the actual function is not in the chosen hypothesis space.

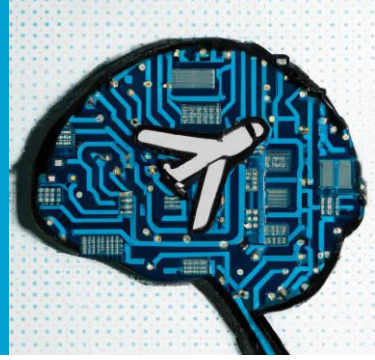
It may also fail because the learning algorithm cannot *find* the right hypothesis in the hypothesis search space.





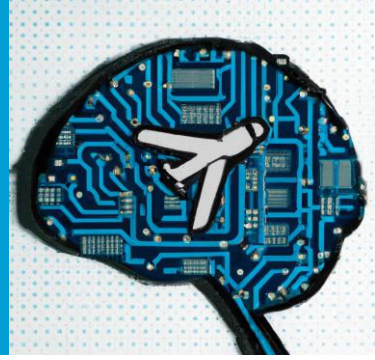
Quiz

Question 1:



Can a perceptron learn the quadratic function $y = ax^2 + bx + c$ when given (x_i, y_i) as input?

Question 1:



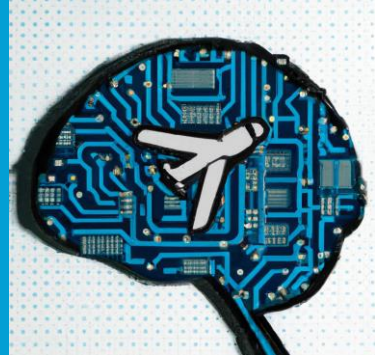
Can a perceptron learn the quadratic function $y = ax^2 + bx + c$ when given (x_i, y_i) as input?

No, not if you provide it with this exact same dataset.

Yes, when changing the input features:

$\mathbf{x} = (x^2, x, 1)$, where $\mathbf{x}$ is the input feature vector.

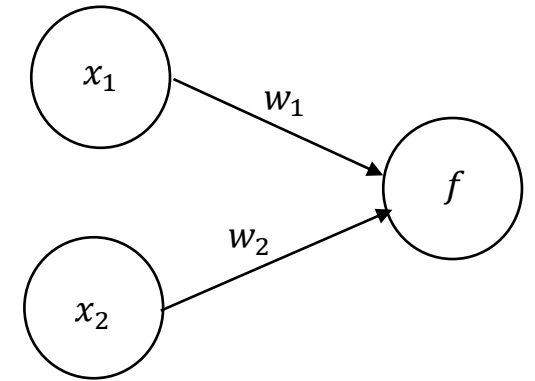
Question 2 – from a previous exam



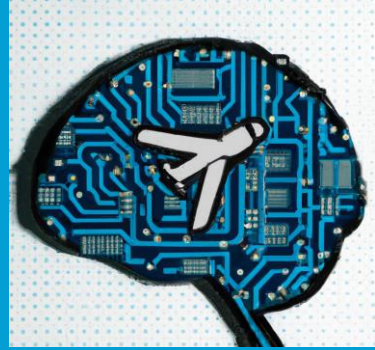
- The simple perceptron neural network on the right has a sine activation function:

$$f(\mathbf{x}) = a \sin(w_1 x_1 + w_2 x_2 + b)$$

What would the Delta learning rule be for weight w_2 , assuming a learning rate α and loss function $L = (f(x) - t)^2$?



$$w_2(i + 1) = w_2(i) + \dots$$



Questions?